CYBERSECURITY HOMELAB REPORT · DETECTION ENGINEERING

Detecting Phishing-Driven Execution: A Wazuh and Splunk SIEM Detection Engineering Homelab

Custom Wazuh rules, a rule-parent (`if_sid`) root-cause investigation, and equivalent Splunk searches, documented with evidence from the lab.

Author	Aigbokhaode Hope Imomoh

Lab dates	27 – 28 October 2026 (Africa/Lagos, UTC+01:00)
Version	1.0 · 28 October 2026
Environment	VirtualBox host-only lab · Ubuntu (Wazuh manager and Splunk Enterprise 9.4.2) · Windows 10 endpoint with Sysmon and the Wazuh agent
Techniques	MITRE ATT&CK T1218.005 (Mshta) · T1059.001 (PowerShell) · T1566 (Phishing) · T1204 (User Execution)
Classification	Educational lab. All activity was harmless test execution on lab-owned virtual machines.

Prepared from the author's own lab screenshots, terminal output and configuration excerpts. Original screenshots are retained unchanged as source evidence; figures in this report are cropped, contrast-adjusted copies. No evidence content has been altered.

Table of Contents

1. Executive Summary 8

2. Introduction and Objectives 9

2.1 Background and motivation 9

2.2 Scope 9

2.3 Objectives and success criteria 9

3. Technical Background 10

3.1 From an action to a dashboard: the detection pipeline 10

3.2 Sysmon Event ID 1 10

3.3 How Wazuh rules work 10

3.4 The key concept in this lab: rule parents 11

3.5 MITRE ATT&CK techniques used 11

4. Lab Environment and Architecture 12

4.1 Machines and addresses 12

4.2 Architecture and data flow 13

4.3 Software and versions 13

5. Prerequisites and Safety 14

5.1 Required resources 14

5.2 Safety and scope controls 14

5.3 Backups and recovery 14

6. Step-by-Step Implementation 15

6.1 Step 1: Review the custom rules as first written 15

6.2 Step 2: Generate harmless test activity on Windows 19

6.3 Step 3: Confirm Sysmon recorded the events 20

6.4 Step 4: Check what the Wazuh dashboard actually shows 21

6.5 Step 5: Add the parent rule to the six detection rules 22

6.6 Step 6: Repair rules 100101 and 100107 23

6.7 Step 7: Validate the configuration, restart the manager and trigger the rule 24

6.8 Step 8: Verify each remaining rule with counts 26

6.9 Step 9: Test the correlation rule 30

6.10 Step 10: View the alerts in the Wazuh dashboard 32

7. Configuration and Detection Logic 40

7.1 The Wazuh rules (reconstructed from the screenshots) 40

7.2 Line-by-line explanation 42

7.3 Rule inventory and status 43

7.4 Equivalent Splunk searches 43

8. Testing and Results 45

8.1 Test matrix 45

8.2 Cross-check between Wazuh and Splunk 45

8.3 What failed, what is unverified 46

9. Evidence and Screenshot Analysis 47

9.1 Image preparation and evidence integrity 47

9.2 Splunk evidence 48

10. Troubleshooting and Root-Cause Analysis 54
10.1 Problem 1: custom rules produced no alerts 54
10.2 Problem 2: correlation rule 100107 rejected 54
10.3 Problem 3: editing mistakes on the command line 55
10.4 Problem 4: forward references 57
10.5 Problem 5: tests that never ran 57
10.6 Unrelated noise 58

11. Security Findings and Risk Analysis 59
11.1 Risk assessment 59

12. Mitigation and Remediation 60

13. What I Learned 61

14. Practical Knowledge and Competencies 62
14.1 Recommended next exercises 62

15. Reproduction Guide 63
15.1 Starting state 63
15.2 Part A: confirm the telemetry 64
15.3 Part B: install the rules 64
15.4 Part C: run the tests (Windows, administrator PowerShell) 64
15.5 Part D: verify 64
15.6 Part E: Splunk (optional) 65
15.7 Troubleshooting 65
15.8 Cleanup and rollback 65

16. Limitations and Future Improvements 66

17. Conclusion 67

Appendix A: Timeline of Work 68

Appendix B: Evidence Register 69

Appendix C: Command History Used in the Lab 72

Appendix D: Glossary and Notes 73

Appendix E: Companion Scripts 74
17.1 Windows test script (administrator PowerShell) 74
17.2 Ubuntu verification script 74

List of Figures

Figure 1. Laboratory architecture and data flow 13
Figure 2. Rules 100104 and 100105 as first written 16
Figure 3. Rules 100106 and 100107 as first written 17
Figure 4. Rules 100510 and 100110 as first written 18
Figure 5. Rules 100110 and 100111 as first written 19
Figure 6. Windows test commands and PowerShell parser errors 20
Figure 7. Sysmon Event ID 1 present on the endpoint 21
Figure 8. Dashboard filtered to rule 92217 only 22
Figure 9. Parent rule added to the detection rules 23
Figure 10. Remaining warning after the first repairs 24
Figure 11. Manager restarted and running 25
Figure 12. First confirmed alert from the custom rules 26
Figure 13. Mistyped commands that did not execute 27
Figure 14. Counts of custom rules in the recent alerts 28
Figure 15. Rule 100510 definition check 29
Figure 16. Rule 100510 count in the full alert file 30
Figure 17. Rule 100107 count of zero before the timed test 31
Figure 18. Rule 100107 final definition 31
Figure 19. Rule 100107 fires: count of 3 32
Figure 20. Dashboard events table: 14 hits for the four rules 33
Figure 21. Alert fields for rule 100105 34
Figure 22. Alert details (1): agent, command line and directory 35
Figure 23. Alert details (2): integrity, hashes and image 36
Figure 24. Alert details (3): parent process 37
Figure 25. Alert details (4): Sysmon provider and raw message 38
Figure 26. Alert details (5): decoder and full log 39
Figure 27. Splunk search for mshta.exe: 10 events over 24 hours 48
Figure 28. mshta.exe results as a table 49
Figure 29. mshta.exe execution times in Splunk 49
Figure 30. PowerShell launches grouped by command line 50
Figure 31. All Sysmon Event 1 records: 1,197 in 24 hours 51
Figure 32. Full Sysmon event in Splunk 52
Figure 33. Correlation search result in Splunk 53
Figure 34. Invalid if_matched_sid reported by the rules test 55
Figure 35. sed syntax error: unknown command '<' 56
Figure 36. Over-broad sed replaced every if_matched_sid 57

List of Tables

Table 1. Objectives, success criteria and outcome 9
Table 2. The detection pipeline and where each stage was verified 10
Table 3. ATT&CK mapping of the custom rules 11

Table 4. Lab components 12
Table 5. Software versions observed 13
Table 6. Test command anatomy 19
Table 7. Anatomy of the perl command 22
Table 8. Validation commands 24
Table 9. Rule elements explained 42
Table 10. Custom rules and verification status 43
Table 11. Wazuh rules and their Splunk counterparts 43
Table 12. Test scenarios and verification status 45
Table 13. mshta.exe executions seen by Splunk and by Wazuh (times are one second apart because alerts are written after analysis) 46
Table 14. Problem 1 analysis 54
Table 15. Findings and confidence 59
Table 16. Mitigations 60
Table 17. Skills demonstrated, with the evidence for each 62
Table 18. Reproduction checklist 63
Table 21. Evidence register: every figure and its source (originals kept unchanged) 69

Abbreviations and Acronyms

ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge, a catalogue of attacker behaviour
EDR / SIEM	Endpoint Detection and Response / Security Information and Event Management
HTA	HTML Application, a script-bearing file executed by `mshta.exe`
IOC	Indicator of Compromise
LOLBin	Living-off-the-land binary: a legitimate operating-system program that attackers misuse
PCRE2	Perl-Compatible Regular Expressions, version 2, the pattern language used in Wazuh rules
SPL	Search Processing Language, Splunk's query language
SOC	Security Operations Centre
Sysmon	Microsoft System Monitor, a Windows driver and service that logs detailed system activity
WAT	West Africa Time, UTC+01:00 (the lab's local time zone)

How to Use This Report

This report can be read in two ways. Choose the path that matches what you want.

If you want to...	Read
Rebuild the lab and get the same alerts	Chapter 5 (prerequisites and safety) → Chapter 15 (Reproduction Guide, with a checklist) → Chapter 7 (the rules) → Appendix E (ready-made scripts). Use Chapter 10 only if something fails.
Understand how the problem was found and fixed	Chapter 3 (pipeline) → Chapter 6 (steps in order, each with a Goal and a Checkpoint) → Chapter 10 (root cause analysis).
Judge the result and the evidence	Chapter 1 (summary) → Chapter 8 (test results) → Chapter 9 (evidence) → Chapter 16 (limitations).
Reuse the Splunk searches	Chapter 7, section 7.4, then Chapter 8, section 8.2.

Conventions

- **Figure N** and **Table N** numbers are the same everywhere; the contents pages list the page of each.
- Commands appear in dark boxes. Run them exactly as shown, one at a time.
- Machine roles: **Ubuntu** = Wazuh manager and Splunk (192.168.56.101); **Windows** = the Windows 10 endpoint with Sysmon and the Wazuh agent (192.168.56.109).
- **Checkpoint** = a result you must see before continuing. **Not tested** / **Not evidenced** mark things this lab did not prove.

CHAPTER 1

Executive Summary

This laboratory built and tested a small detection system for the endpoint behaviour that typically follows a successful phishing attack: a trusted Windows program such as `mshta.exe` or `powershell.exe` is launched from a user's `Downloads` folder, or several such launches occur in quick succession. A Windows 10 virtual machine produced the activity, Sysmon recorded it, the Wazuh agent forwarded the events to a Wazuh manager on an Ubuntu virtual machine, and custom rules in `local_rules.xml` turned them into alerts. The same data was later sent to Splunk Enterprise 9.4.2, where equivalent searches were written.

The security problem. Phishing is a common first step in intrusions, and the stage after the click (execution) is where defenders have a second chance to notice. Process-creation telemetry from Sysmon contains the information needed, but it only helps if rules are written, loaded and verified to fire.

What happened. The custom rules were written but initially produced no alerts. Investigation showed that Sysmon was logging correctly (Event ID 1 present on the endpoint) and that the Wazuh dashboard was receiving other Sysmon-based alerts, so the fault lay in the custom rules. The rules had no parent rule (`<if_sid>`), and the correlation rule 100107 referenced rules incorrectly. Adding `<if_sid>61603</if_sid>` to six rules and correcting the correlation rule made the rules fire. Several mistakes made while editing the file on the command line are documented in Chapter 10 rather than hidden.

4 of 7
custom detection rules verified firing end to end (100105, 100107, 100111, 100510)

6
rules repaired by adding the missing parent rule 61603

3
rules not yet tested (100104, 100106, 100110)

1,197
Sysmon Event 1 records seen in Splunk (24 h), 10 of them `mshta.exe`

Verified outcomes. The Wazuh dashboard showed 14 alerts for rules 100105, 100107, 100111 and 100510 on agent DESKTOP-T6T1N2V (Figure 20). The raw alert file `alerts.json` recorded rule 100111 at level 12 (Figure 12) and counted 5 alerts for rule 100510 and 3 for rule 100107. In Splunk, the `mshta.exe` search returned the same test executions, and a correlation search using `streamstats` returned the repeated-execution bursts (Figure 33).

What is not claimed. No real phishing email was sent and GoPhish was not run. Rules 100104 (Office application spawning a shell), 100106 (drop-path parent command line) and 100110 (browser spawning a shell) were not triggered. Splunk saved alerts and triggered-alert evidence were not captured. These gaps are listed in Chapters 8 and 16.

CHAPTER 2

Introduction and Objectives

2.1 Background and motivation

Security teams cannot stop every phishing message, so they also watch what happens on the endpoint after a user opens a malicious attachment or link. Attackers frequently chain legitimate Windows programs (called living-off-the-land binaries, or LOLBins) to avoid dropping obvious malware. `mshta.exe` runs HTML Applications, a technique catalogued by MITRE as T1218.005, and PowerShell is the most commonly abused scripting host (T1059.001). Seeing either program launched from a user's `Downloads` folder is a meaningful signal.

2.2 Scope

In scope: a three-machine VirtualBox lab on a host-only network, Sysmon process-creation telemetry (Event ID 1), custom Wazuh rules, the Wazuh dashboard, and Splunk searches over the same events. Out of scope: real malware, internet-facing systems, email delivery infrastructure and automated response.

2.3 Objectives and success criteria

Table 1. Objectives, success criteria and outcome

#	Objective	Success criterion	Result
O1	Confirm Sysmon generates process-creation events on the Windows 10 endpoint	Event ID 1 records for the test commands in the Sysmon operational log	**Met** (Figure 7)
O2	Write custom Wazuh rules for phishing-style execution	Rules present in `local_rules.xml` and loaded without errors	**Met** after repairs (Figures 9, 11, 12)
O3	Make rules fire on harmless test commands	Alert visible in `alerts.json` and the dashboard	**Met** for 100105, 100107, 100111, 100510
O4	Correlate repeated activity	Rule 100107 fires on two matching events within 180 s	**Met** (count of 3, Figures 17–19)
O5	Reproduce the detection logic in Splunk	Searches returning the same test events	**Partly met**: searches verified; alerts not saved or evidenced
O6	Test the full phishing delivery chain (GoPhish)	Browser-launched shell triggers rule 100110	**Not met**: not performed

CHAPTER 3

Technical Background

3.1 From an action to a dashboard: the detection pipeline

Beginners often treat "the SIEM did not alert" as one problem. It is really a chain of separate stages, and a failure at any stage looks the same on the dashboard. This lab was solved by testing each stage in turn.

Table 2. The detection pipeline and where each stage was verified

Stage	Plain meaning	In this lab	How it was checked
1. Event generation	The operating system or a tool records that something happened	Sysmon logs Event ID 1 (process created) to the Windows event log	PowerShell `Get-WinEvent` on the endpoint (Figure 7)
2. Collection	An agent reads the log	Wazuh agent reads the Sysmon operational channel	Other Sysmon-based alerts (rule 92217) already appeared (Figure 8)
3. Forwarding	The agent sends the event to the server	Agent 006 (192.168.56.109) to manager (192.168.56.101)	Alerts show `agent.name` `DESKTOP-T6T1N2V`
4. Decoding	The server splits raw text into named fields	Decoder `windows_eventchannel` yields `win.eventdata.image` and similar	Document details (Figures 22–26)
5. Rule evaluation	Rules compare fields against patterns	Built-in rules and custom rules in `local_rules.xml`	`wazuh-analysisd -t` and test runs
6. Alert generation	A matching rule writes an alert	`/var/ossec/logs/alerts/alerts.json`	`grep` and `uniq -c` on the file
7. Dashboard display	The alert is indexed and searchable	Wazuh dashboard, Threat Hunting, Events	Filter `rule.id: (100105 OR 100107 OR 100111 OR 100510)`

3.2 Sysmon Event ID 1

Sysmon (System Monitor) is a free Microsoft tool that records detailed activity. Event ID 1 is written whenever a process starts and contains the program path (`Image`), its command line, the working directory, the user, the integrity level, file hashes, and the parent process. Parent and child together are what make behaviour suspicious: `powershell.exe` starting `mshta.exe` tells a story that neither event tells alone.

3.3 How Wazuh rules work

A Wazuh rule has an ID, a severity level (0–15) and conditions. `<field>` compares a decoded field with a value or a regular expression (`type="pcre2"`). `<if_sid>` makes a rule a child of another rule, so it is only checked after the parent has matched. `frequency` and `timeframe` turn a rule into a correlation rule that fires after a given number of matches in a given number of seconds, and `<if_matched_sid>` names the rule whose matches are counted. Custom rule IDs must be 100000 or above.

3.4 The key concept in this lab: rule parents

Windows Sysmon events are processed through a chain of built-in rules. The built-in rule for Sysmon Event 1 (process creation) is rule 61603. Custom rules that describe Event 1 behaviour are expected to attach to that chain with `<if_sid>61603</if_sid>`. In this lab, rules without that line never produced an alert, and adding it fixed them (Chapter 10). The precise internal evaluation order was not independently traced with `wazuh-logtest` in verbose mode, so the explanation is supported by the experimental result and by Wazuh's documented rule-tree behaviour, not by a separate trace.

3.5 MITRE ATT&CK techniques used

Table 3. ATT&CK mapping of the custom rules

Technique	Name	Rule(s)	Why it matters
T1218.005	System Binary Proxy Execution: Mshta	100105	`mshta.exe` runs HTA files and inline script, so it can launch a payload without a new executable
T1059.001	Command and Scripting Interpreter: PowerShell	100510	PowerShell is the dominant scripting tool for post-exploitation
T1566 / T1566.001	Phishing / Spearphishing Attachment	100104, 100111	Execution from `Downloads` or an Office parent is typical after a phishing delivery
T1204.001 / .002	User Execution: Link / File	100104, 100110	The user's click starts the chain; a browser or Office app is the parent

CHAPTER 4

Lab Environment and Architecture

4.1 Machines and addresses

Addresses and names below are taken from the Wazuh and Splunk screenshots unless stated otherwise. All are private lab addresses.

Table 4. Lab components

Component	Details	Evidence
Ubuntu VM ("Ubuntu_SnapShot")	Hostname `Ubuntu`, user `hope`, IP 192.168.56.101. Runs the Wazuh manager and dashboard (`http://192.168.56.101`). Wazuh logtest reports v4.14.7. Editor: GNU nano 8.7.1.	Figures 2–5, 11, 20
Splunk Enterprise 9.4.2	Web interface reached at `127.0.0.1:8000` from the Ubuntu desktop, so Splunk appears to run on the Ubuntu VM. Index `sysmon`.	Figures 27–33
Windows 10 VM	Computer name `DESKTOP-T6T1N2V.HopeLabs.Local`, user `aigbo`, IP 192.168.56.109, Wazuh agent ID 006, Sysmon (provider Microsoft-Windows-Sysmon, schema version 5), `mshta.exe` file version 11.00.19041.5794.	Figures 6, 7, 22–26
Windows Server 2019 VM	Domain `HopeLabs.Local`, host-only IP 192.168.56.115, per the author's notes. Not visible in the screenshots, but a domain user (`HOPELABS\HR-STAFF1`) appears in Sysmon events in Splunk.	Author's notes; Figure 33
Network	VirtualBox: adapter 1 host-only (192.168.56.0/24), adapter 2 NAT, per the author's notes.	Author's notes
Time zone	Africa/Lagos (WAT, UTC+01:00). Wazuh and Splunk display local time; the Sysmon `UtcTime` field is UTC, hence one hour earlier.	Figures 25, 29, 32

4.2 Architecture and data flow

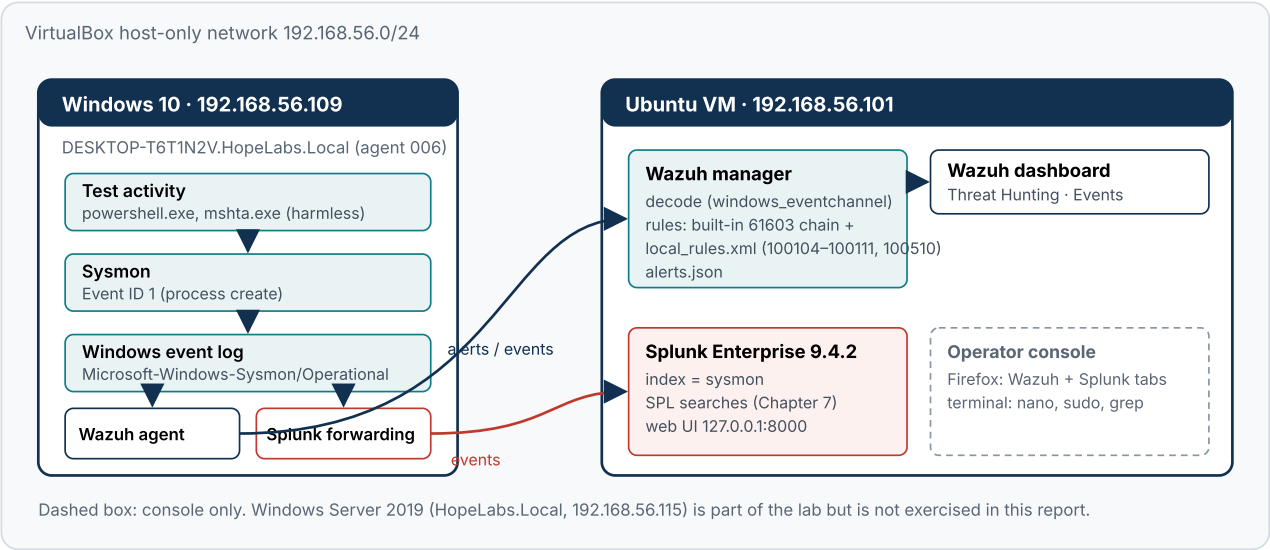


Figure 1. Laboratory architecture and data flow

Step: Chapter 4 **Source:** drawn for this report from the evidence listed in Chapter 4

Two parallel paths carry the same Sysmon events: the Wazuh agent to the Wazuh manager (blue) and Splunk forwarding to Splunk (red). The Ubuntu VM hosts both analysis platforms. Component names and addresses are from the screenshots; the diagram simplifies network ports, which were not captured.

4.3 Software and versions

Table 5. Software versions observed

Software	Version / detail	Where seen
Oracle VirtualBox	Windows 10 and Ubuntu guests (window titles show [Running])	Figures 2–6
Wazuh	wazuh-logtest v4.14.7; indices named `wazuh-alerts-4.x-2026.10.27` ; decoder `windows_eventchannel`	Figures 16, 22, 26
Sysmon	Provider Microsoft-Windows-Sysmon, event schema version 5	Figures 25, 32
Splunk Enterprise	9.4.2; sourcetype `WinEventLog:Microsoft-Windows-Sysmon/Operational`	Figures 27–33
GNU nano	8.7.1	Figures 2–5
Windows 10	`mshta.exe` file version 11.00.19041.5794 (WinBuild.160101.0800)	Figure 23

CHAPTER 5

Prerequisites and Safety

5.1 Required resources

- Host computer able to run three virtual machines (the lab VMs are separate; Splunk and Wazuh share the Ubuntu VM, so give it enough memory, ideally 8 GB, for both).
- Ubuntu VM with Wazuh manager, indexer and dashboard installed and healthy.
- Windows 10 VM with Sysmon installed and the Wazuh agent enrolled (agent shows Active).
- Splunk Enterprise (trial or free licence) and a forwarding method for Windows events (optional part of the lab).
- A user with `sudo` rights on Ubuntu and administrator rights on Windows.

5.2 Safety and scope controls

- **Only harmless commands were used.** The `mshta.exe` test was `javascript:closed()` / `javascript:close()`, which opens and closes the HTML host without any payload. PowerShell tests only printed text and changed directory.
- **Isolation.** The machines sit on a VirtualBox host-only network. Do not run these tests on production or personal systems.
- **No real malware or real phishing mail.** No credentials or personal data appear in the evidence; IPs are private lab addresses.
- **Edit a copy first.** Rule files can break all custom detection if malformed (see Chapter 10). Back up before editing.

5.3 Backups and recovery

Before the command-line edits, a backup of `local_rules.xml` was created (`local_rules.xml.bak`). It later proved essential: it preserved the original value of rule 100101 after an over-broad `sed` command changed it (Figures 35, 36). VM snapshots of both machines are recommended before and after configuration changes; the Ubuntu VM is named "Ubuntu_SnapShot", which suggests a snapshot workflow was already in use.

```
sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak
# rollback if needed:
sudo cp /var/ossec/etc/rules/local_rules.xml.bak /var/ossec/etc/rules/local_rules.xml
sudo systemctl restart wazuh-manager
```

> **Note.** The rollback block above is standard practice. Only the backup command appears in the lab history; a rollback was not needed.

CHAPTER 6

Step-by-Step Implementation

> **Note. How to use this chapter.** It records the lab in the order it happened, including the mistakes, so you can see how the problem was diagnosed. Every step has a Goal and a Checkpoint: do not move on until the checkpoint is true. If you only want a clean build, follow the Reproduction Guide in Chapter 15, which gives the corrected sequence without the detours.

The steps follow the order in which work happened on 27–28 October. Times come from the screenshots. Where a standard procedure is described that is not captured in the evidence, it is labelled as such.

6.1 Step 1: Review the custom rules as first written

Goal: Read exactly what is in `local_rules.xml` before changing anything. **Checkpoint:** You can see the four rule groups and notice there is no `<if_sid>` line (Figures 2–5).

Where: Ubuntu VM terminal, `sudo nano /var/ossec/etc/rules/local_rules.xml`. **Why:** the rules did not alert, so the first task was to read what was actually in the file. Four rule groups are present: phishing/office_exploit (100104–100107), windows,sysmon,powershell (100510) and gophish_lab,phishing_simulation (100110, 100111).

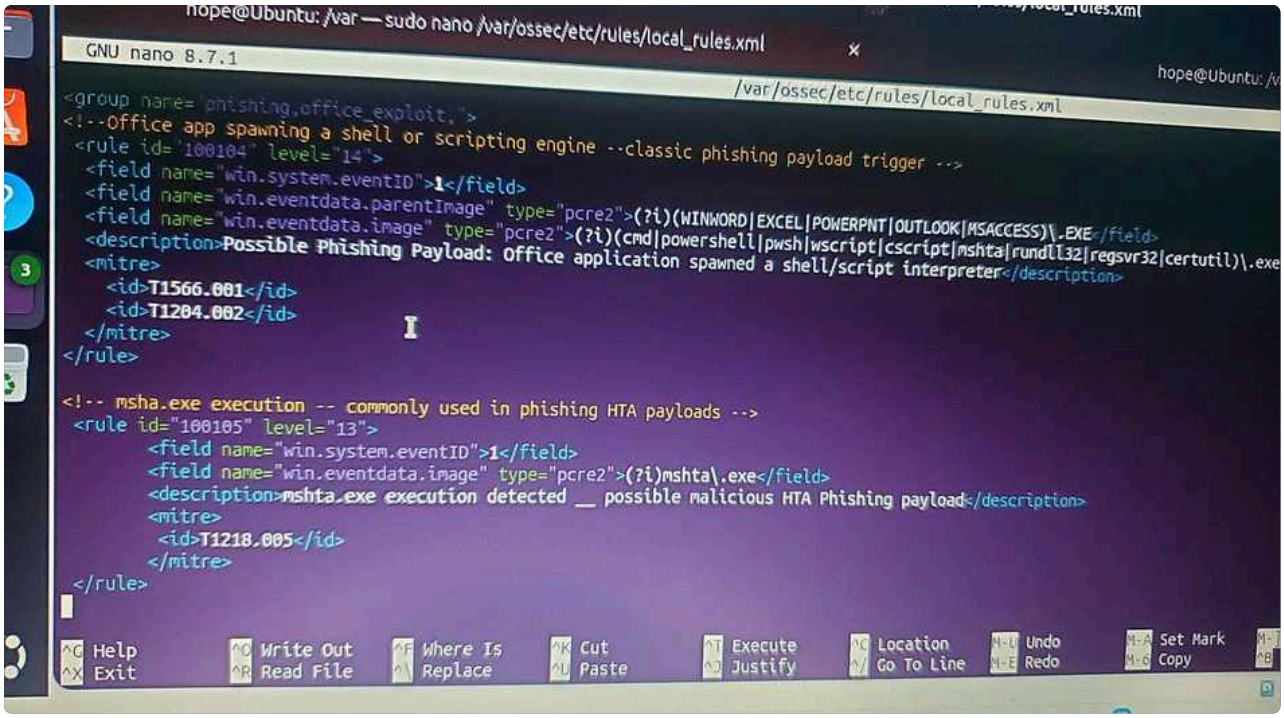


Figure 2. Rules 100104 and 100105 as first written

Step: 1 **Source:** 0168da3c (10:51)

Shows: The `phishing,office_exploit` group. Rule 100104 (level 14) matches an Office parent (WINWORD, EXCEL, POWERPNT, OUTLOOK, MSACCESS) spawning a script host; rule 100105 (level 13) matches `mshta.exe`. Both test `win.system.eventID` equal to 1 and map to ATT&CK IDs.
Demonstrates: Neither rule contains an `<if_sid>` line. This is the missing element found later.
Limitations: Photograph of the screen at an angle; text is legible but not pixel-sharp. The comment in the file spells "msha.exe", a typo in the comment only.

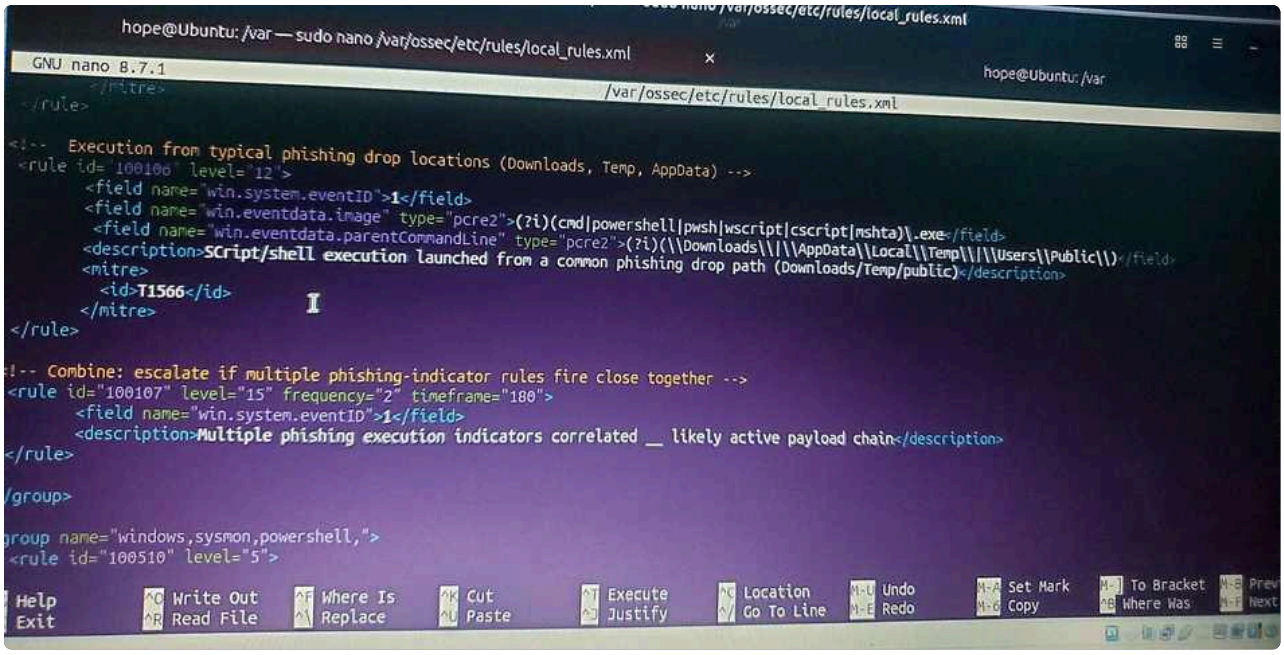


Figure 3. Rules 100106 and 100107 as first written

Step: 1 **Source:** 5fbc4433 (10:51)

Shows: Rule 100106 (level 12) matches a shell or script host whose *parent* command line contains Downloads, AppData\Local\Temp or Users\Public. Rule 100107 (level 15, `frequency="2"`, `timeframe="180"`) is the correlation rule, but contains only an `eventID` field and no `<if_matched_sid>`.

Demonstrates: Rule 100107 had no rule to count, so it could not behave as a correlation rule as intended. Rule 100106 tests the *parent* command line, so it will not match the later tests, where the parent is a plain `powershell.exe`.

Limitations: Photograph at an angle.

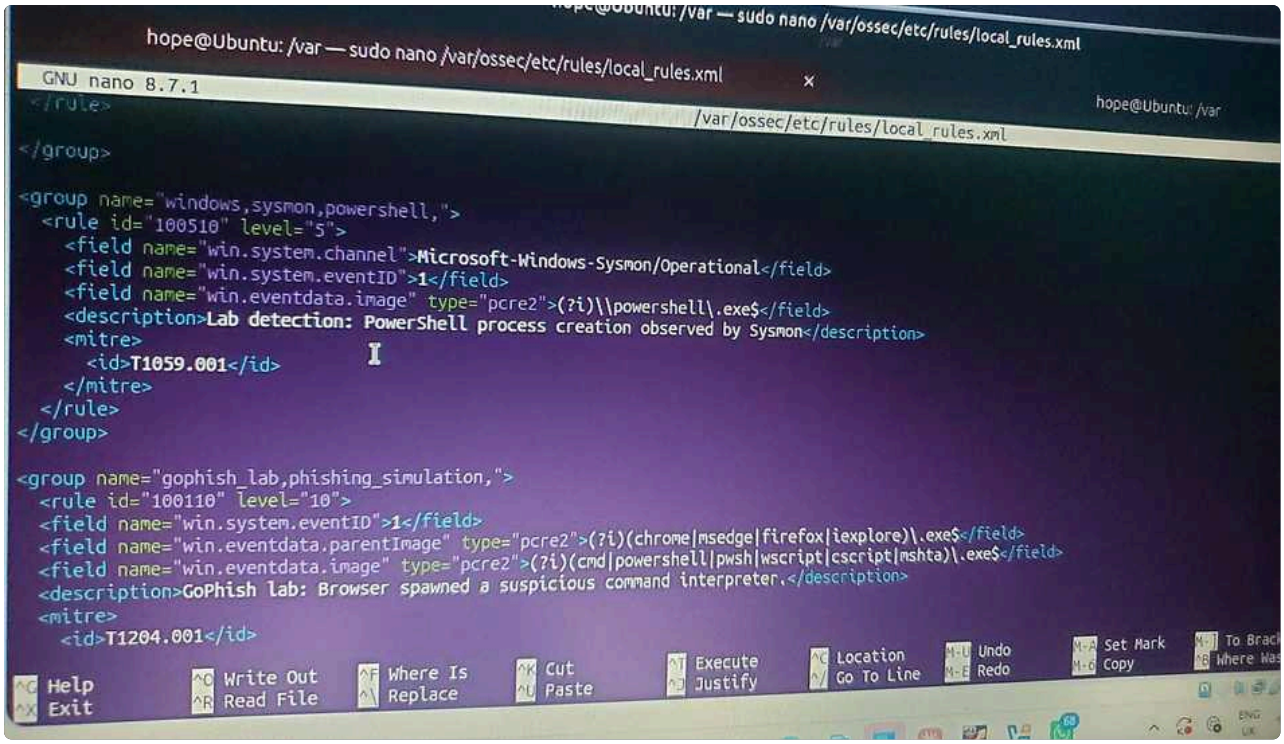


Figure 4. Rules 100510 and 100110 as first written

Step: 1 **Source:** 9a6b46fa (10:51)

Shows: Rule 100510 (level 5) matches the Sysmon channel, Event ID 1 and an image path ending in `\powershell.exe`. Rule 100110 (level 10, group `gophish_lab`) matches a browser parent (chrome, msedge, firefox, iexplore) launching a command interpreter.
Demonstrates: Again no `<if_sid>`. Rule 100510 checks the channel explicitly, but still needs a parent.
Limitations: Photograph at an angle.

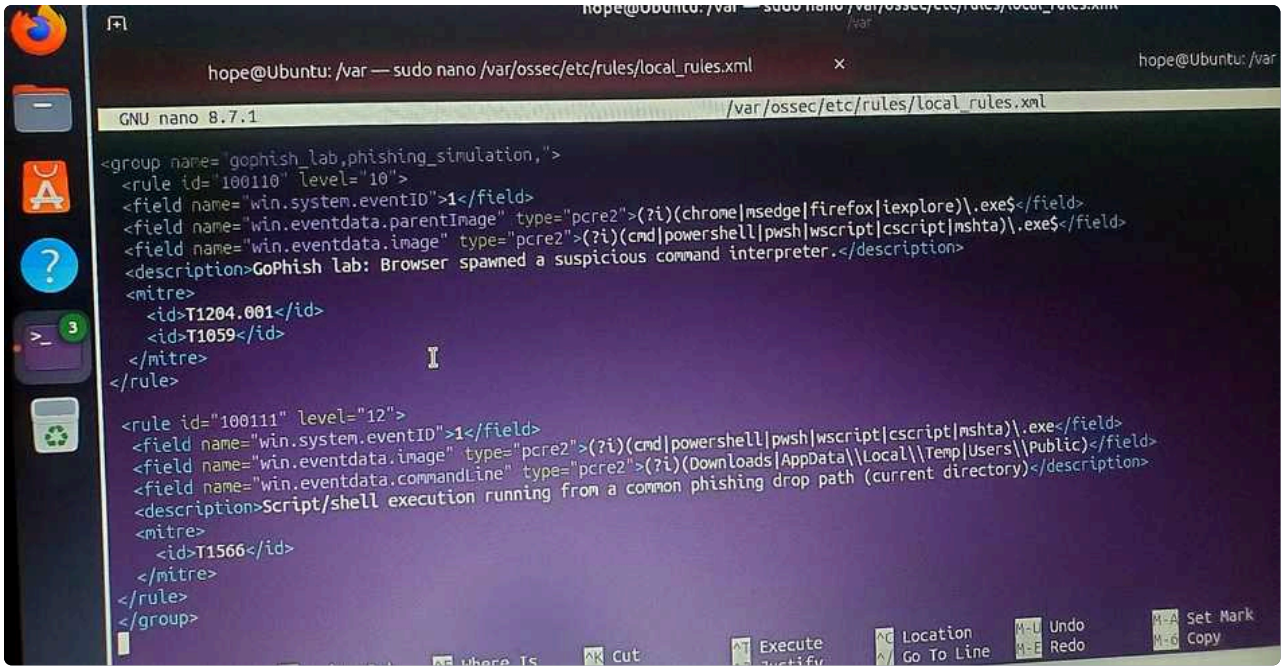


Figure 5. Rules 100110 and 100111 as first written

Step: 1 **Source:** 839324d6 (10:52)

Shows: Rule 100111 (level 12) matches a shell or script host whose own *command line* contains Downloads, AppData\Local\Temp or Users\Public, mapped to T1566. This is the rule that later fired on the test commands.
Demonstrates: Same missing parent. Rule 100111 tests the process's own command line, which is why the `cd ...\Downloads` test triggers it.
Limitations: Photograph at an angle.

6.2 Step 2: Generate harmless test activity on Windows

Goal: Create Sysmon Event 1 records the rules should match. **Checkpoint:** Commands run without red parser errors.

Where: administrator PowerShell on the Windows 10 VM. **Why:** to create Sysmon Event 1 records whose fields the rules should match.

```
powershell.exe -Command "cd $env:USERPROFILE\Downloads; Write-Host 'Lab test v8'"
mshta.exe "javascript:close()"   # lab history shows javascript:closed() variants
```

Table 6. Test command anatomy

Part	Meaning
`powershell.exe -Command "..."`	Start a new PowerShell process and run the quoted text. The new process creates a Sysmon Event 1 whose command line contains `Downloads` (rule 100111).
`cd $env:USERPROFILE\Downloads`	Change directory to the current user's Downloads folder.
`Write-Host 'Lab test v8'`	Print a harmless marker that identifies the test run.
`mshta.exe "javascript:close()"`	Start the HTML Application host with an inline script that does nothing harmful. Sysmon logs the process creation regardless of what the script does (rule 100105).

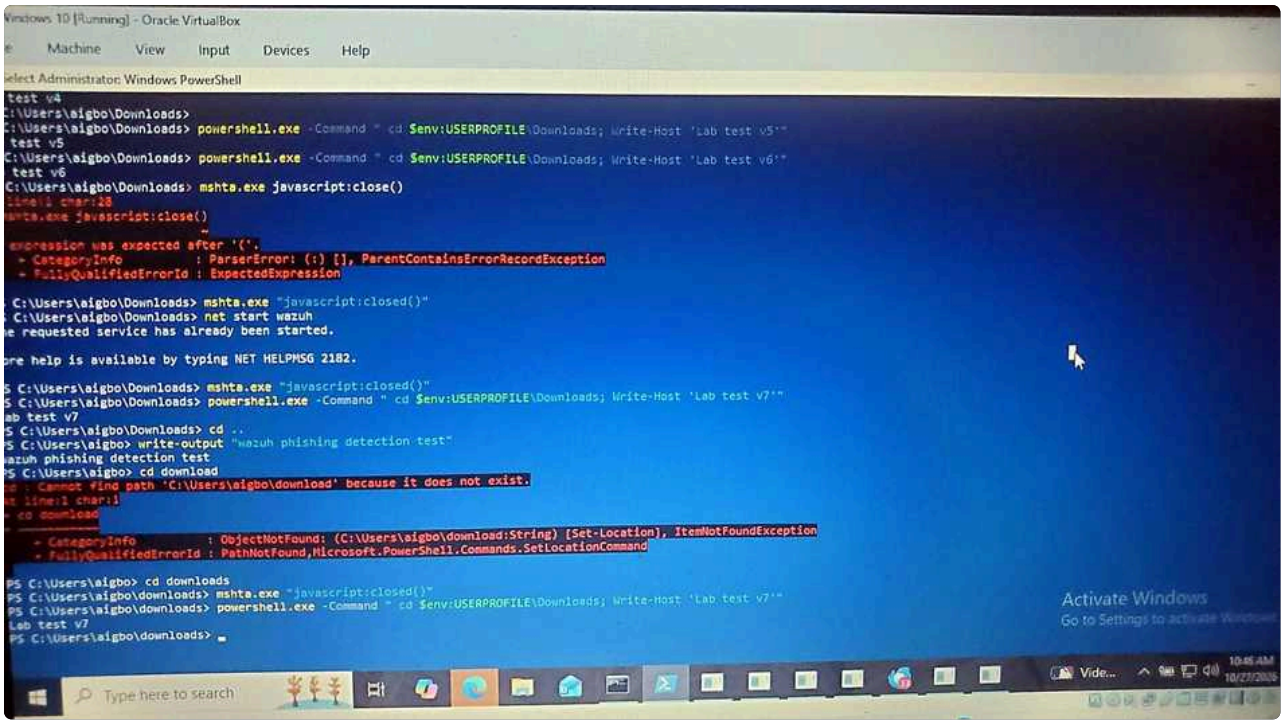


Figure 6. Windows test commands and PowerShell parser errors

Step: 2 **Source:** fc586c77 (10:46)

Shows: An administrator PowerShell session running `Lab test v5`, `v6` and `v7` (each from the Downloads folder) and several `mshta.exe` attempts. Red text shows parser errors where `mshta.exe javascript:close()` was typed without quotes; the working form uses quotes.
Demonstrates: Test activity was generated, and the first attempts show how quoting matters in PowerShell. Errors are left in the figure on purpose.
Limitations: Photograph of a Windows PowerShell window; the VirtualBox frame and taskbar are partially visible. Timestamp in the taskbar reads 10:46 AM.

6.3 Step 3: Confirm Sysmon recorded the events

Goal: Prove stage 1 of the pipeline works. **Checkpoint:** `Get-WinEvent` lists Event ID 1 entries with current timestamps (Figure 7). If it is empty, stop: fix Sysmon before touching Wazuh.

Where: Windows PowerShell. **Why:** if Sysmon does not log Event 1, no rule can ever fire. This isolates stage 1 of the pipeline.

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=1} -MaxEvents 5 |
Select TimeCreated, Message | Format-List
```

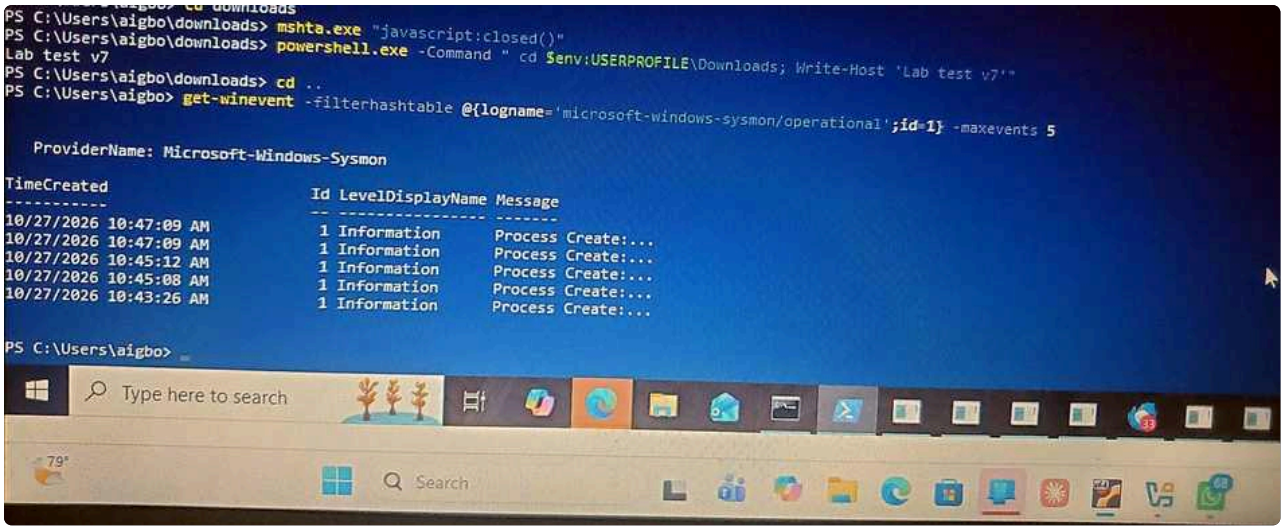


Figure 7. Sysmon Event ID 1 present on the endpoint

Step: 3 **Source:** 91edd109 (10:50)

Shows: `Get-WinEvent` output listing Event ID 1 ("Process Create") at 10:47:09, 10:45:12, 10:45:08 and 10:43:26 on 10/27/2026.
Demonstrates: Event generation works: the Windows machine records process creation for the test commands, so the fault is downstream.
Limitations: Only the first characters of each message are visible. Event times match the test commands the author ran.

6.4 Step 4: Check what the Wazuh dashboard actually shows

Goal: Prove collection works (stages 2–3). **Checkpoint:** Other Sysmon-based alerts for the agent appear (Figure 8).

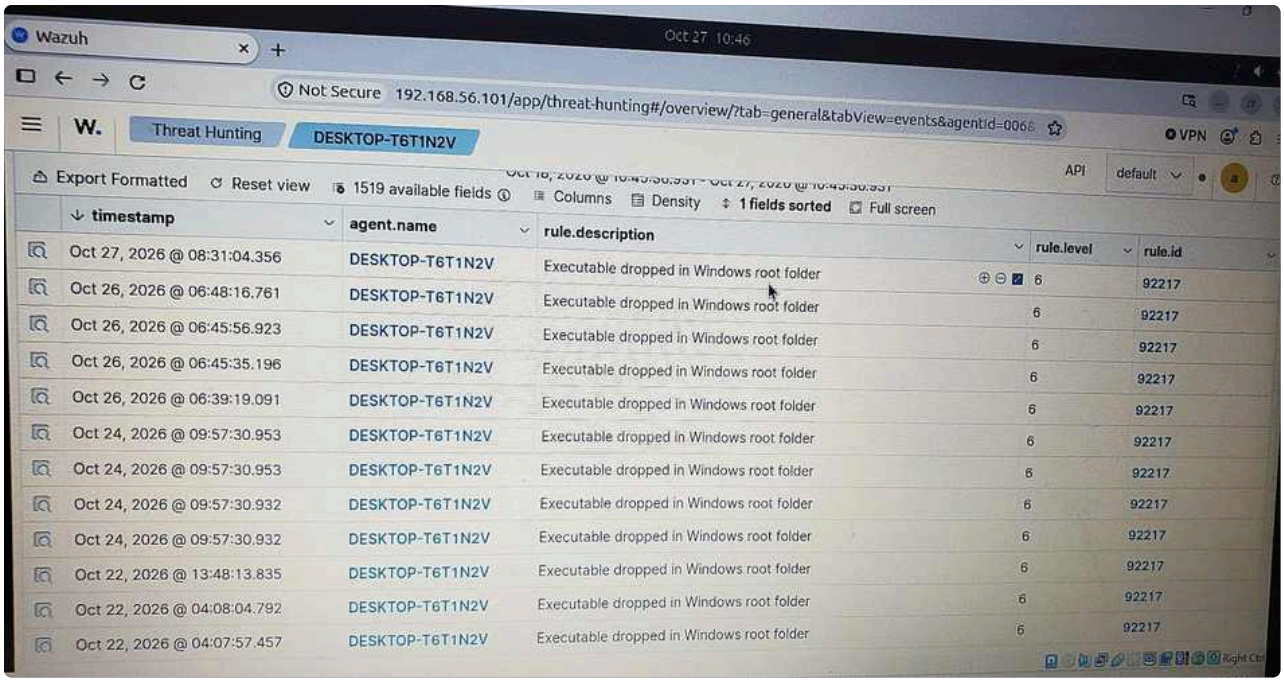


Figure 8. Dashboard filtered to rule 92217 only

Step: 4 **Source:** a4bebdec (10:46)

Shows: Wazuh Threat Hunting Events for agent DESKTOP-T6T1N2V showing only rule 92217 ("Executable dropped in Windows root folder", level 6), the latest at 08:31 on 27 Oct.
Demonstrates: The agent and the collection path are alive, because built-in Sysmon-based alerts arrive. No alert from the custom rules appears for the new tests.
Limitations: A filter on rule 92217 was active, so this screenshot cannot show alerts from other rules. It supports "collection works", not "nothing else arrived".

6.5 Step 5: Add the parent rule to the six detection rules

Goal: Attach the rules to the Sysmon Event 1 rule chain. **Checkpoint:** `grep` shows `<if_sid>61603</if_sid>` under each of the six rules (Figure 9).

Why: the evidence pointed to the custom rules themselves. A parent rule attaches them to Wazuh's Sysmon Event 1 chain. The parent ID, 61603, was confirmed empirically: after adding it, the rules fired.

```
sudo perl -0pi -e 's/(<rule id="(?:100104|100105|100106|100110|100111|100510)"[^>]*>)/$1\n
<if_sid>61603<\/if_sid>/g' /var/ossec/etc/rules/local_rules.xml
```

Table 7. Anatomy of the perl command

Part	Meaning
`sudo`	Run as root; the rules file is owned by root (a plain `grep` earlier returned "Permission denied").
`perl -0pi -e`	`-0` reads the whole file at once, `-p` prints the result, `-i` edits the file in place, `-e` supplies the script.
`s/(<rule id="(?:...)"[^>]*>)/$1\n <if_sid>61603<\/if_sid>/g`	For each listed rule's opening tag, keep the tag (`$1`) and add a new line with the parent rule after it.

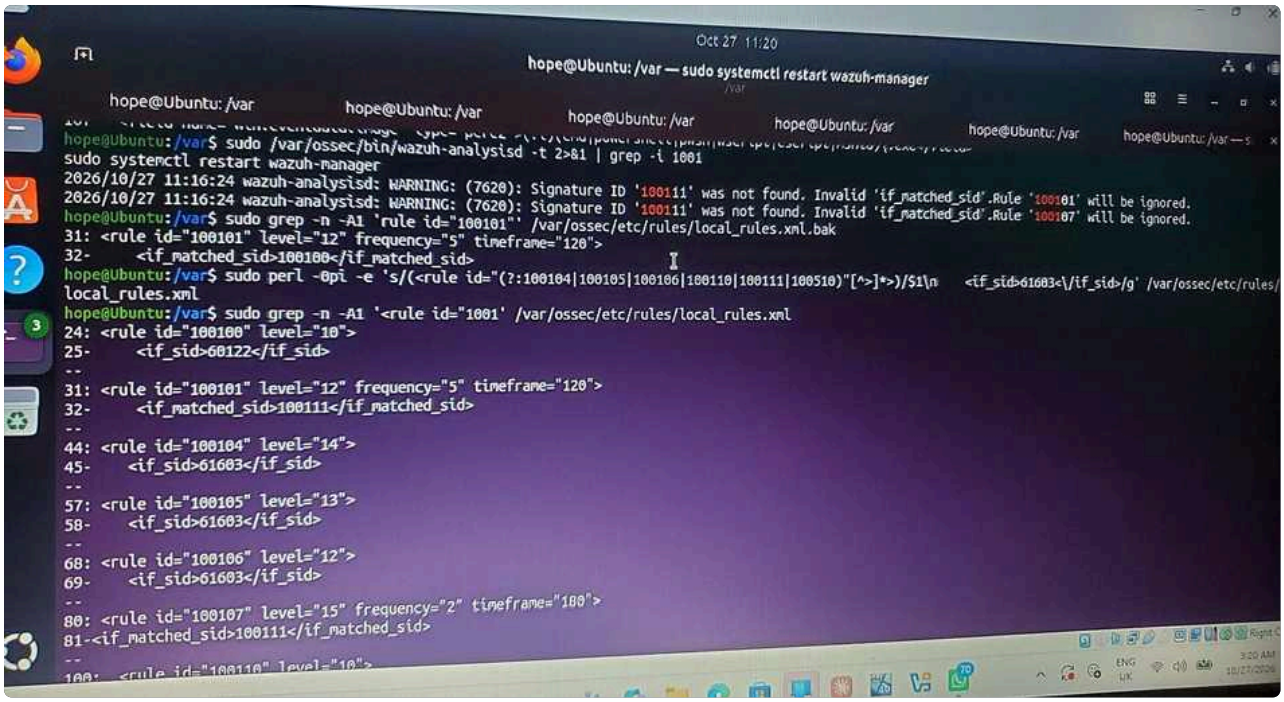


Figure 9. Parent rule added to the detection rules

Step: 5 **Source:** 4c7e2e02 (11:20)

Shows: `grep -n -A1 '<rule id="1001'` output showing `<if_sid>61603</if_sid>` directly under rules 100104, 100105 and 100106, along with the earlier warnings about rules 100101 and 100107.
Demonstrates: The perl edit applied. Line 32 (rule 100101) still shows a wrong value left by an earlier command; that is fixed in Step 6.
Limitations: Only the first part of the file is visible in this photograph; 100110, 100111 and 100510 were checked in later output (Figure 15).

6.6 Step 6: Repair rules 100101 and 100107

Goal: Remove invalid or forward references so every rule loads. **Checkpoint:** `wazuh-analysisd -t | grep -i 1001` prints nothing (Figures 10, 11).

Two rules still referred to a rule that did not exist at that point in the file. Rule 100101 was restored to its original value (`100100`, taken from the backup), and rule 100107 was pointed at rule 100105, which is defined earlier in the file and is the rule that fired in the test.

```
sudo sed -i '/<rule id="100101"/,/<\/rule>/s|<if_matched_sid>100111</if_matched_sid>|
<if_matched_sid>100100</if_matched_sid>|' /var/ossec/etc/rules/local_rules.xml
sudo sed -i '/<rule id="100107"/,/<\/rule>/s|<if_matched_sid>100111</if_matched_sid>|
<if_matched_sid>100105</if_matched_sid>|' /var/ossec/etc/rules/local_rules.xml
```

The address range `/<rule id="100101"/,/<\/rule>/` limits each substitution to one rule. This corrected the over-broad command described in Chapter 10.

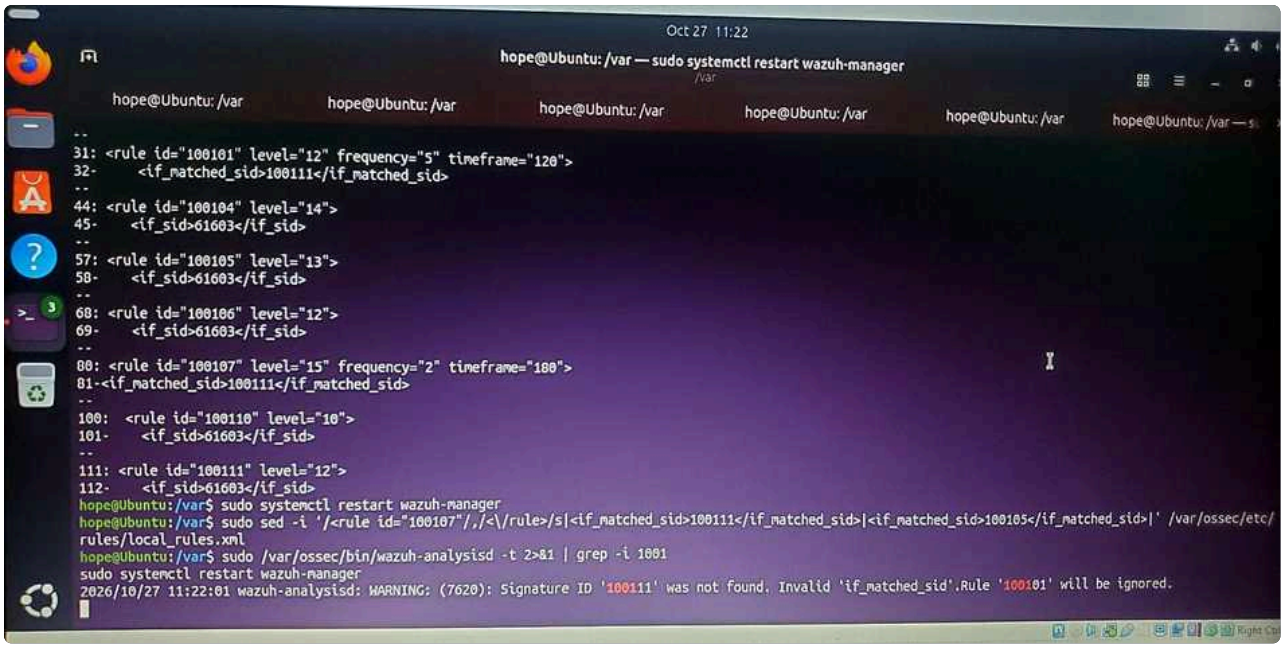


Figure 10. Remaining warning after the first repairs

Step: 6 **Source:** eb5f2ad3 (11:22)

Shows: After the edit, `wazuh-analysisd -t` output still reports Signature ID 100111 not found for rule 100101. Rule 100107 now has `<if_matched_sid>100105</if_matched_sid>`.
Demonstrates: Progress check: only the 100101 warning remained, and it was fixed by the first command above.
Limitations: Photograph at an angle.

6.7 Step 7: Validate the configuration, restart the manager and trigger the rule

Goal: Load the rules and get the first alert. **Checkpoint:** Manager is `active (running)` and an alert for rule 100111 appears in `alerts.json` (Figures 11, 12).

```
sudo /var/ossec/bin/wazuh-analysisd -t 2>&1 | grep -i 1001   # expect no output
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager --no-pager | head -n 5
```

Table 8. Validation commands

Command	Purpose	Expected result
`wazuh-analysisd -t`	Test mode: load all rules and report errors without running	No lines mentioning rule IDs 1001xx
`... | grep -i 1001`	Filter the output (which also contains many unrelated IOC warnings) to this lab's rules	Empty output means the custom rules loaded cleanly
`systemctl restart wazuh-manager`	Reload rules into the running service	Prompt returns with no error
`systemctl status wazuh-manager`	Confirm service is running	`Active: active (running)`

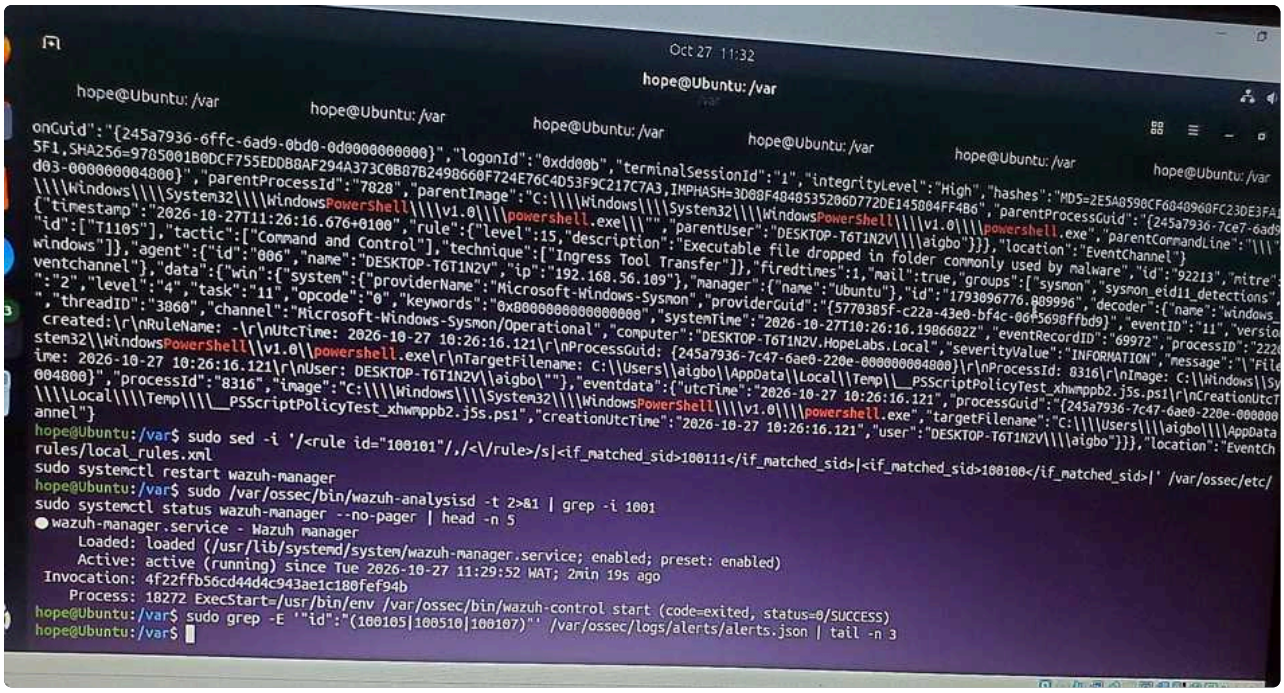


Figure 11. Manager restarted and running

Step: 7 **Source:** 83eb9efc (11:32)

Shows: Terminal output showing `wazuh-manager.service` as `active (running)` since 11:29:52 and the rules test returning no 1001xx warning lines.
Demonstrates: The repaired rule file loads and the manager is healthy.
Limitations: The warnings about the `malicious-ioc` lists (rules 99901–99920) are unrelated to this lab and were left in place (Chapter 10).

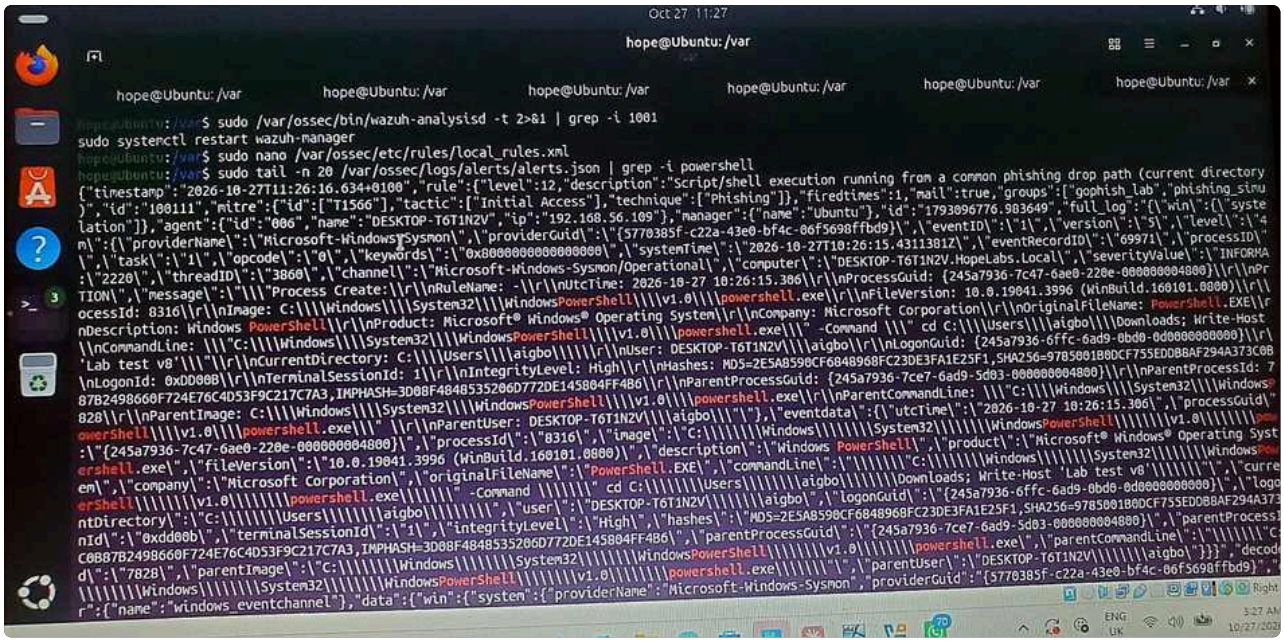


Figure 12. First confirmed alert from the custom rules

Step: 7 **Source:** a196c9b0 (11:27)

Shows: `alerts.json` contains an alert from rule 100111, level 12, description "Script/shell execution running from a common phishing drop path (current directory)", groups `gophish_lab` and `phishing_simulation`, ATT&CK T1566, agent 006 (DESKTOP-T6T1N2V, 192.168.56.109), with the test command `Lab test v8` in the Downloads directory.
Demonstrates: The repair worked: a custom rule now fires on a harmless test command. This was the first verified detection.
Limitations: The raw JSON is very dense in a photograph; a second alert (built-in rule 92213) from PowerShell's own temporary policy-test script is also present and is a benign side effect.

6.8 Step 8: Verify each remaining rule with counts

Goal: Confirm rules 100105 and 100510 fire, using counts over the whole file. **Checkpoint:** Count for 100510 is above 0 (Figure 16); 100105 appears in the counts.

Rather than reading the log by eye, counts from `alerts.json` were used.

```
sudo tail -n 200 /var/ossec/logs/alerts/alerts.json | grep -oE '"id":"1001[0-9]+"' | sort | uniq -c
sudo grep -c '"id":"100510"' /var/ossec/logs/alerts/alerts.json
sudo grep -c '"id":"100107"' /var/ossec/logs/alerts/alerts.json
```

`grep -o` prints only the matching part, `sort | uniq -c` counts duplicates, and `grep -c` counts matching lines. Counting is more reliable than scrolling through JSON.

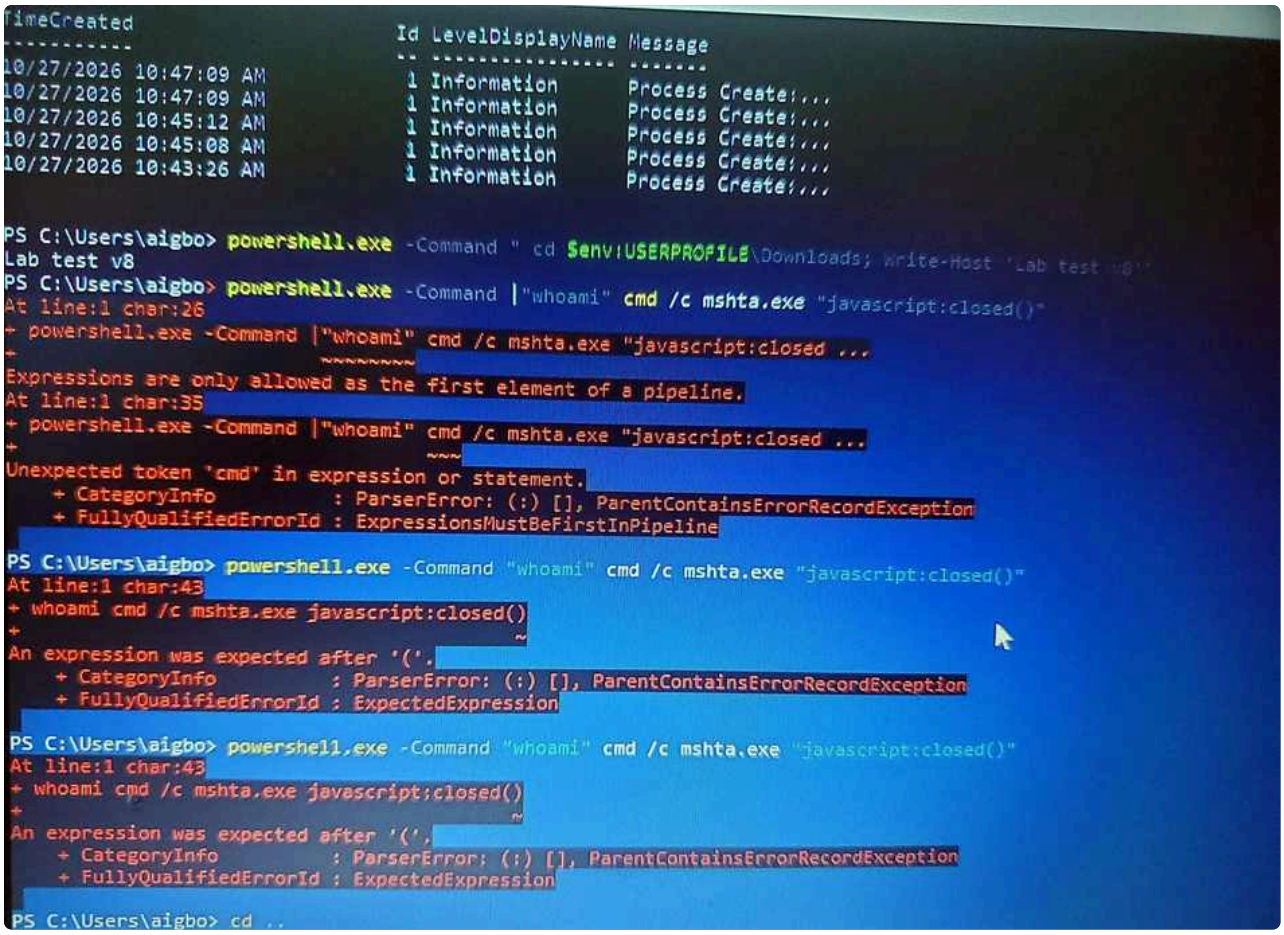


Figure 13. Mistyped commands that did not execute

Step: 8 **Source:** f2d317e8 (11:45)

Shows: PowerShell errors "Expressions are only allowed as the first element of a pipeline" and "An expression was expected after '('": two commands were pasted on one line and the quoting was wrong. The Sysmon event list at the top shows that the run produced no new matching events.
Demonstrates: A reminder that a missing alert can simply mean the test never ran. The test was repeated as separate, correctly quoted lines.
Limitations: Photograph; the lower part of the PowerShell window is cropped by the camera angle.

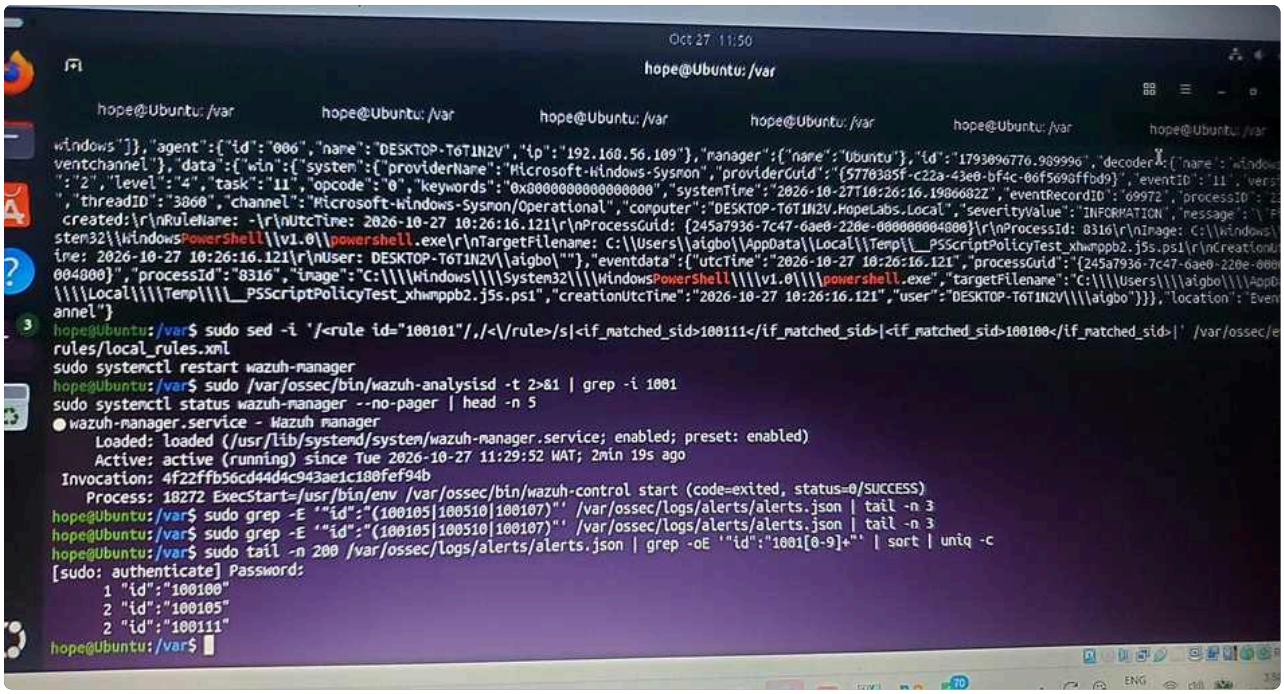


Figure 14. Counts of custom rules in the recent alerts

Step: 8 **Source:** d39c5648 (11:50)

Shows: Output of the `uniq -c` pipeline: rule 100100 once, rule 100105 twice, rule 100111 twice.
Demonstrates: Rules 100105 and 100111 are confirmed firing; 100510 is missing from the last 200 lines at this point.
Limitations: The last 200 lines are dominated by recent high-level alerts, so a rule that fired earlier can be absent from this window.

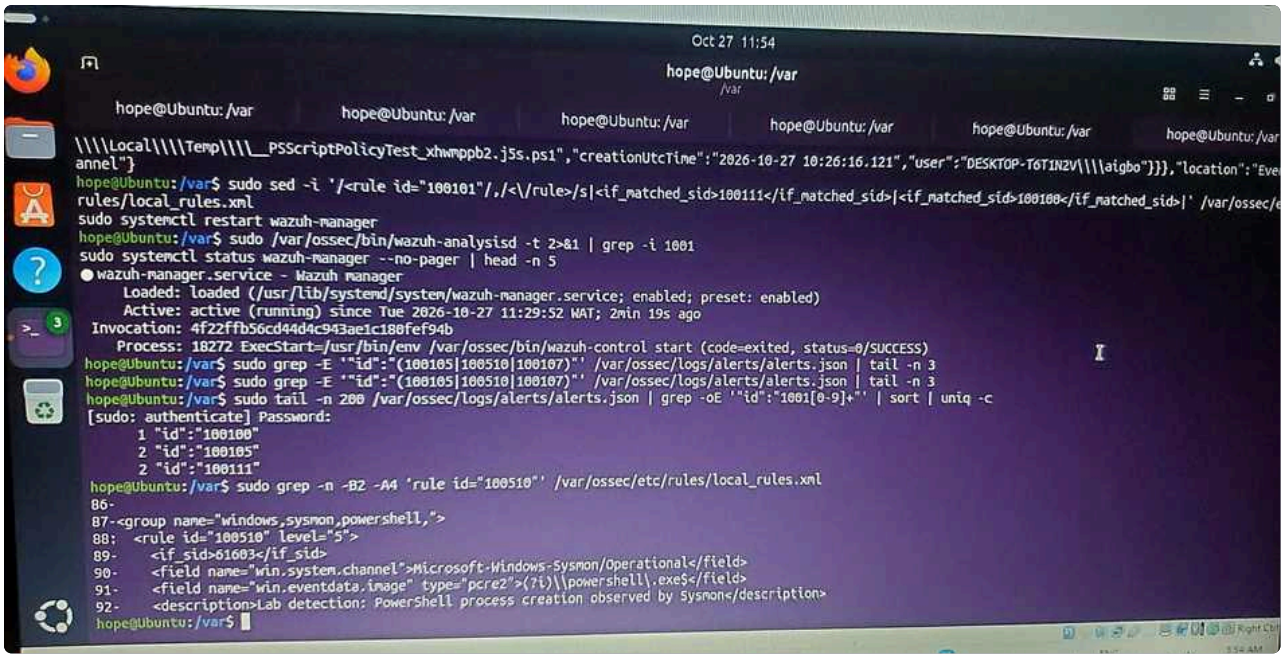


Figure 15. Rule 100510 definition check

Step: 8 **Source:** b787a062 (11:54)

Shows: `grep -B2 -A4 'rule id="100510"'` shows the rule inside group `windows,sysmon,powershell,` with `<if_sid>61603</if_sid>` , the Sysmon channel field and the PowerShell image regex.

Demonstrates: The rule is defined correctly, so the absence from the 200-line window was a sampling effect, not a rule fault.

Limitations: Photograph; only the relevant lines are legible.

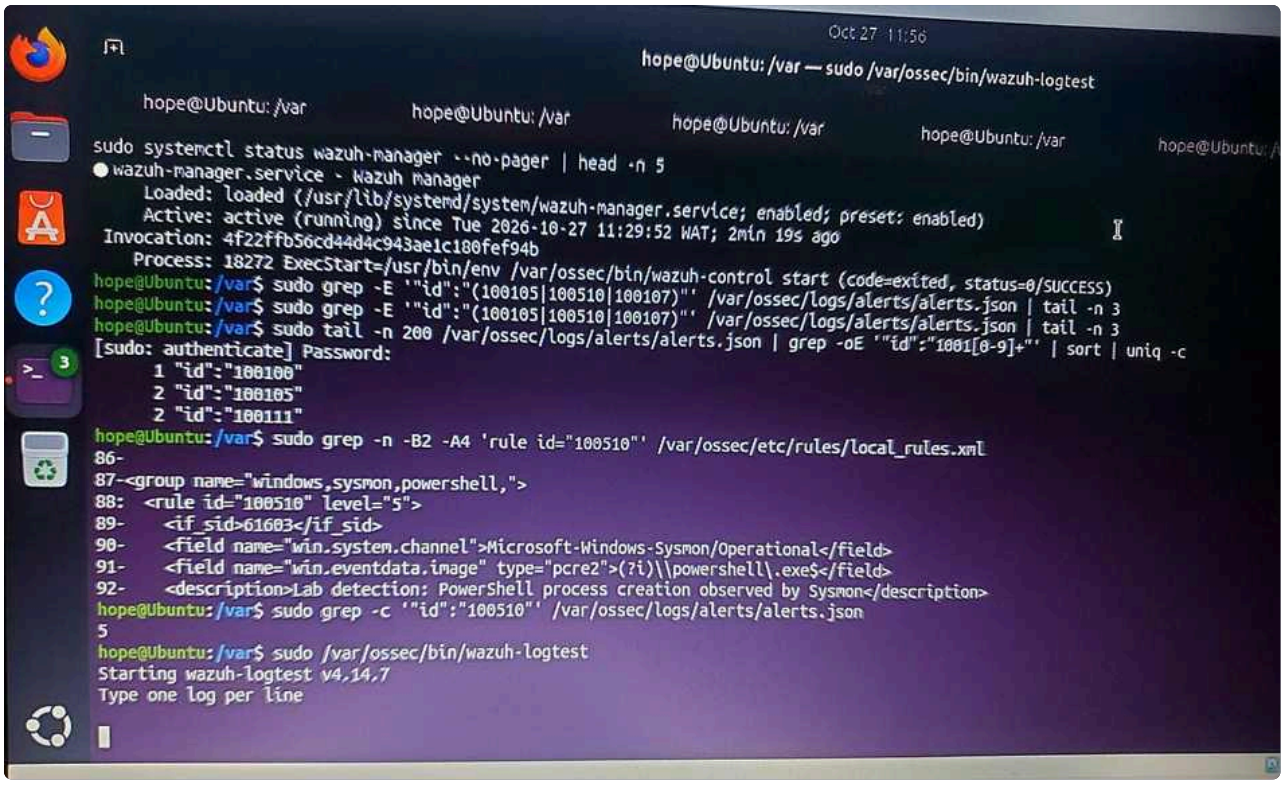


Figure 16. Rule 100510 count in the full alert file

Step: 8 **Source:** 20a12e1d (11:56)

Shows: `grep -c` on the complete `alerts.json` returns **5** for rule 100510. The same photograph also shows `wazuh-logtest` started (v4.14.7) after the count.
Demonstrates: Rule 100510 fires: five alerts recorded. Together with 100105 and 100111, three rules are confirmed.
Limitations: Counts cover the whole file, which includes earlier tests.

6.9 Step 9: Test the correlation rule

Goal: Confirm rule 100107 fires when two mshta.exe runs fall inside 180 seconds. **Checkpoint:** Count for 100107 rises from 0 to 3 (Figures 17–19).

Rule 100107 needs two rule 100105 matches within 180 seconds. A first count returned 0, a mis-pasted command landed in `wazuh-logtest` ("No decoder matched", expected for a non-log line), the rule definition was rechecked, and the test was repeated with two `mshta.exe` runs in quick succession.

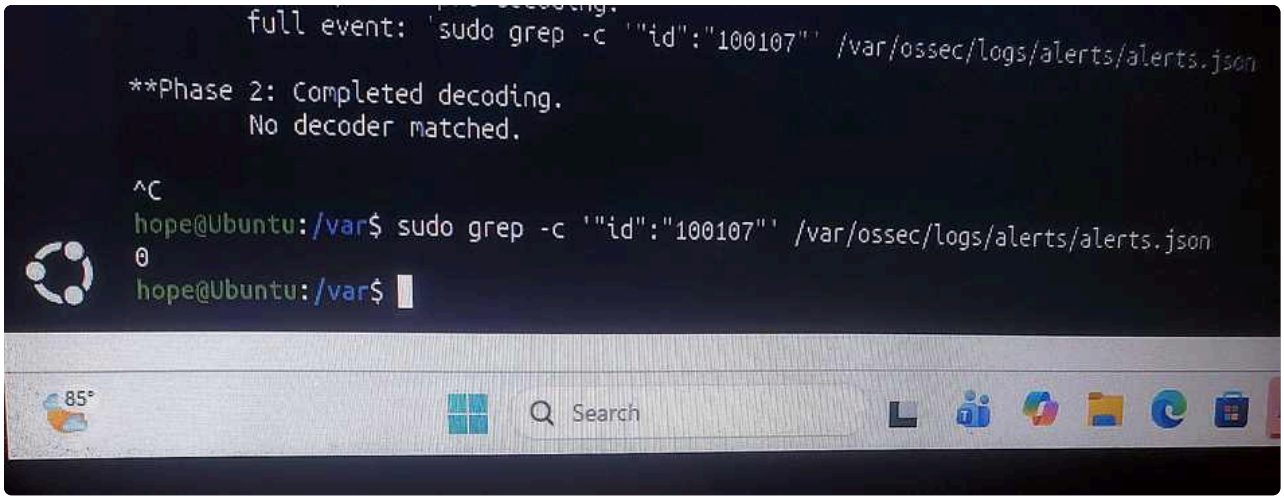


Figure 17. Rule 100107 count of zero before the timed test

Step: 9 **Source:** 3e6ea7a0 (11:58)

Shows: `sudo grep -c '"id":"100107"'` returned 0 after two earlier `mshta` runs that were too far apart.
Demonstrates: The rule had not fired yet. This is a valid negative result: the earlier runs did not satisfy `timeframe="180"`.
Limitations: A zero count alone cannot distinguish a timing problem from a broken rule; the definition was checked next (Figure 18).

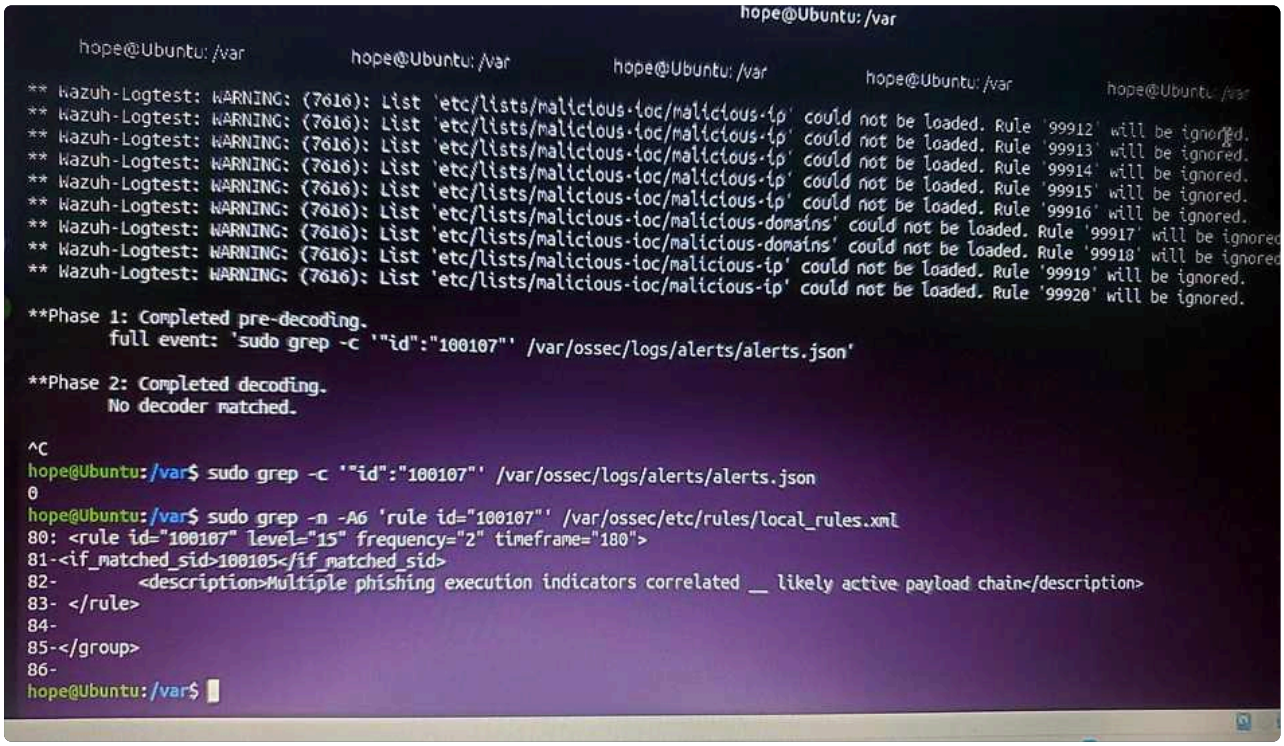


Figure 18. Rule 100107 final definition

Step: 9 **Source:** 1093cae9 (11:59)

Shows: The rule has level 15, `frequency="2"`, `timeframe="180"`, `<if_matched_sid>100105</if_matched_sid>` and the description "Multiple phishing execution indicators correlated __ likely active payload chain".
Demonstrates: The definition is correct; the earlier zero count was a timing issue.
Limitations: Photograph; the rest of the file is cropped.

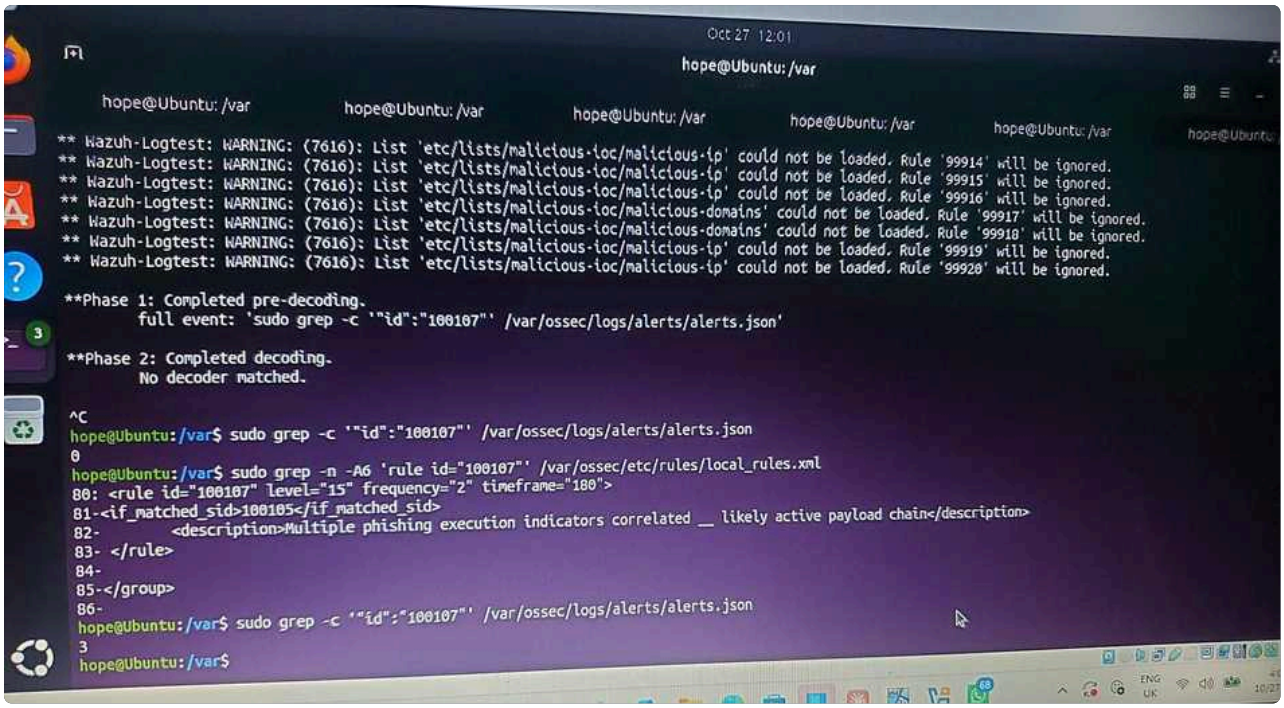


Figure 19. Rule 100107 fires: count of 3

Step: 9 **Source:** 94a04de7 (12:01)

Shows: After two `mshta.exe` runs within seconds, `grep -c` returns **3**.
Demonstrates: The correlation rule works as designed.
Limitations: Three alerts, not two, because additional `mshta.exe` runs from the same test burst each raised the correlation alert (cross-checked against Splunk in Chapter 8).

6.10 Step 10: View the alerts in the Wazuh dashboard

Goal: Confirm the alerts are searchable in the dashboard. **Checkpoint:** Filter shows rules 100105, 100107, 100111, 100510 (Figure 20) and the alert fields match the test command (Figures 21–26).

Where: Wazuh dashboard, Threat Hunting, agent DESKTOP-T6T1N2V, Events tab. Time range last 9 days, query `rule.id: (100105 OR 100107 OR 100111 OR 100510)`.

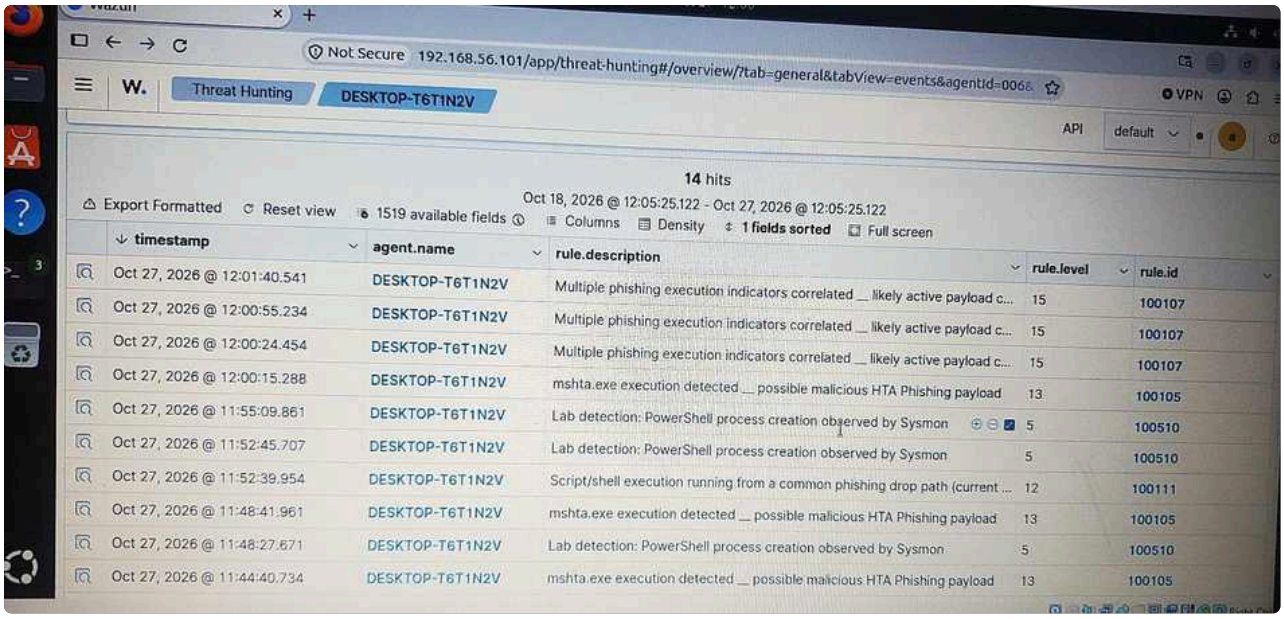


Figure 20. Dashboard events table: 14 hits for the four rules

Step: 10 **Source:** 5c9da716 (12:05)

Shows: 14 hits for agent DESKTOP-T6T1N2V, time range 18–27 Oct. Visible rows include rule 100107 (level 15) at 12:01:40, 12:00:55 and 12:00:24; 100105 (level 13) at 12:00:15, 11:48:41 and 11:44:40; 100510 (level 5) at 11:55:09, 11:52:45 and 11:48:27; 100111 (level 12) at 11:52:39.
Demonstrates: The SIEM front end displays the custom alerts with their levels and descriptions.
Limitations: Photograph of a monitor; the table is legible but small.

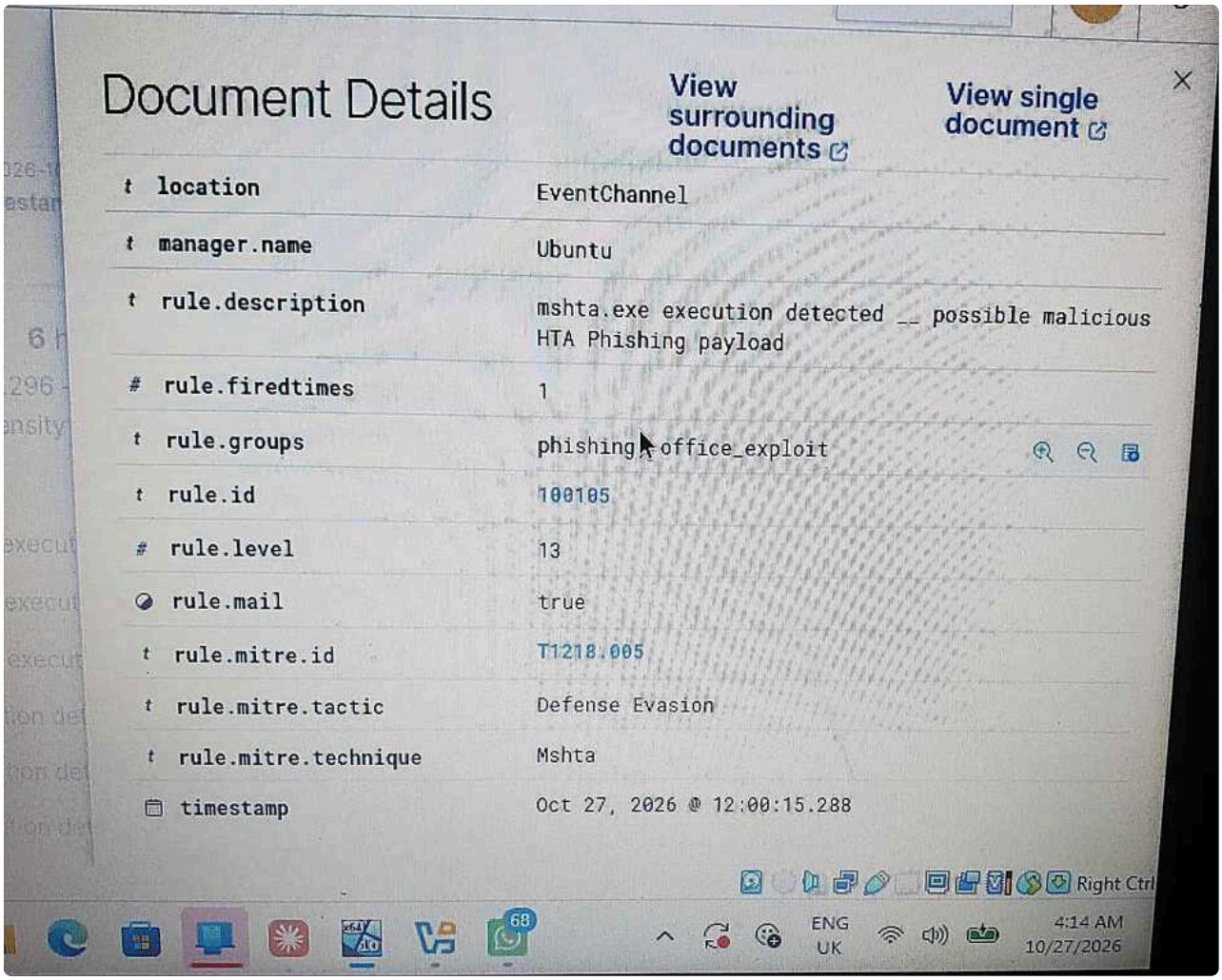


Figure 21. Alert fields for rule 100105

Step: 10 **Source:** 1214031916733... (12:14)

Shows: Document details for a rule 100105 alert: description "mshta.exe execution detected __ possible malicious HTA Phishing payload", groups `phishing, office_exploit`, rule level 13, fired times 1, ATT&CK `T1218.005`, tactic Defense Evasion, technique Mshta, timestamp Oct 27, 2026 @ 12:00:15.288.
Demonstrates: The ATT&CK mapping declared in the rule is carried into the alert.
Limitations: Photograph at a steep angle; cropped for legibility.

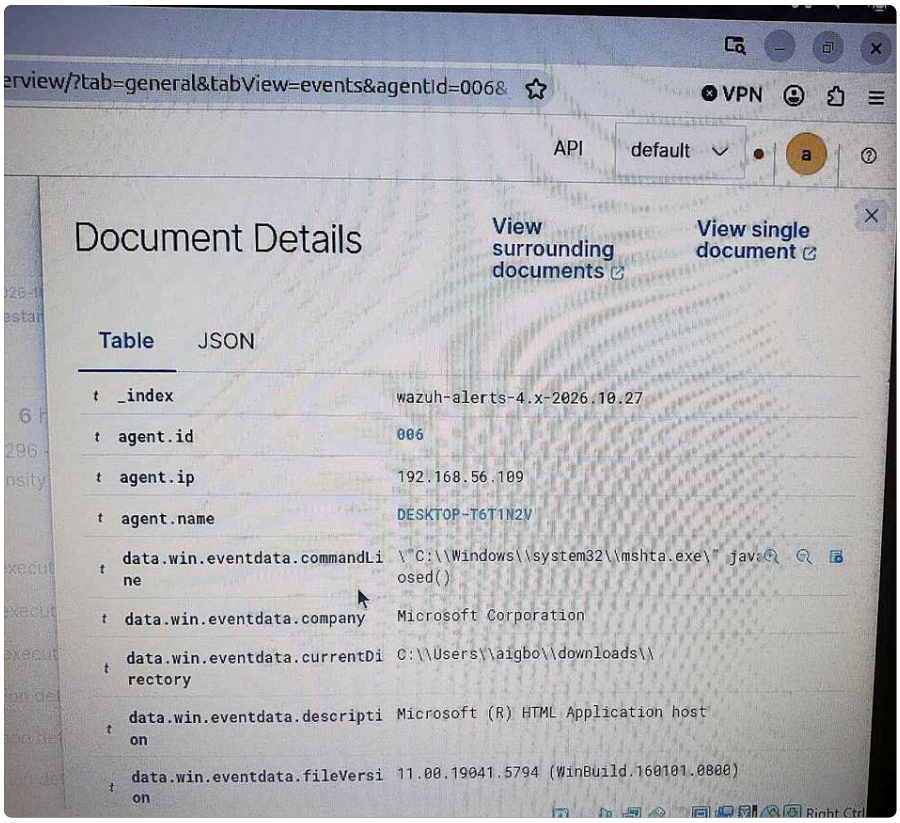


Figure 22. Alert details (1): agent, command line and directory

Step: 10 **Source:** 20874ea7 (12:08)

Shows: Document details table: index `wazuh-alerts-4.x-2026.10.27`, agent ID 006, IP 192.168.56.109, agent name DESKTOP-T6T1N2V, `commandLine` `"C:\Windows\system32\mshta.exe" javascript:closed()`, company Microsoft Corporation, current directory `C:\Users\aigbo\downloads\`, description Microsoft (R) HTML Application host.
Demonstrates: The decoded fields show exactly what ran and where.
Limitations: Photograph at an angle; portrait orientation.

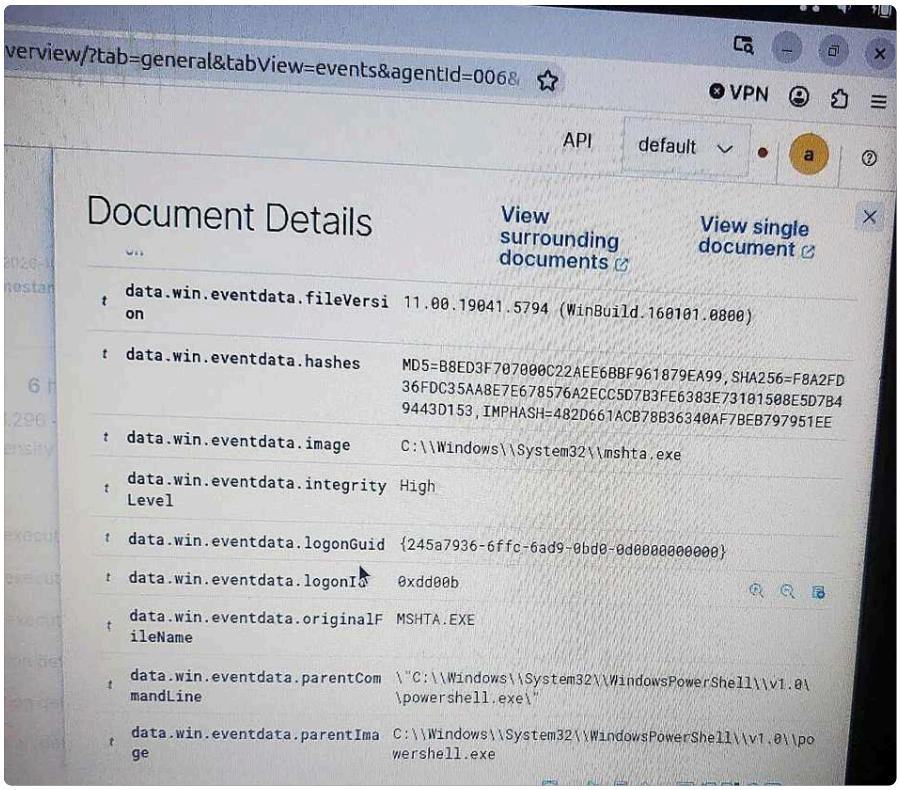


Figure 23. Alert details (2): integrity, hashes and image

Step: 10 **Source:** a61c6de4 (12:08)

Shows: `fileVersion` 11.00.19041.5794, SHA256 and IMPHASH hashes for `mshta.exe`, image `C:\Windows\System32\mshta.exe`, integrity level High, original file name MSHTA.EXE, logon ID `0xdd00b`.
Demonstrates: A genuine Windows binary was run, with high integrity because an administrator shell was used.
Limitations: Hash values are transcribed from photographs in the report text only where clearly legible.

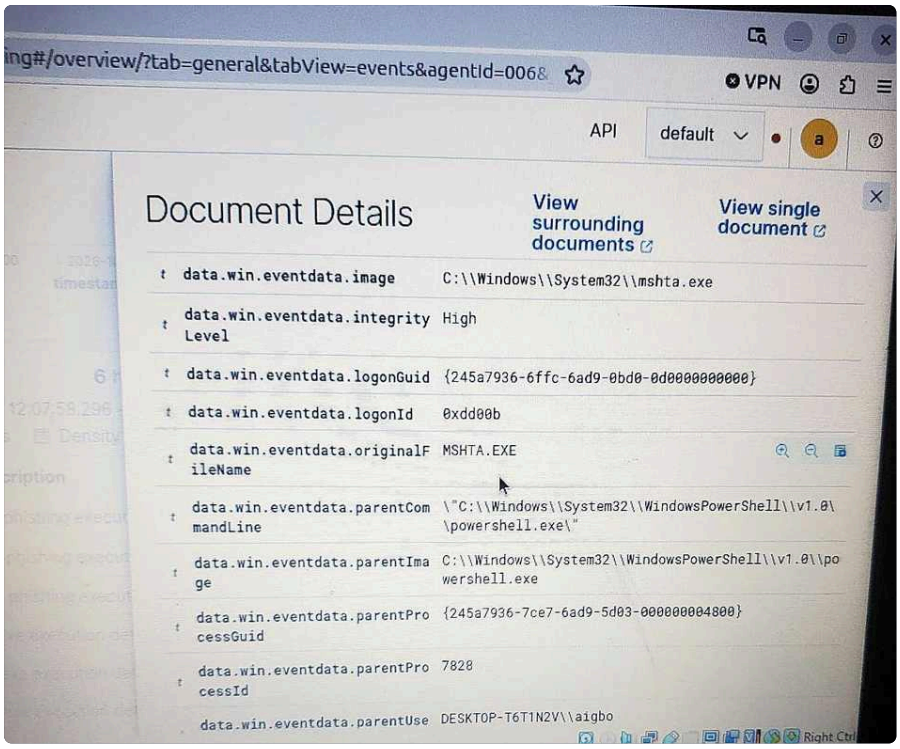


Figure 24. Alert details (3): parent process

Step: 10 **Source:** d7f9ed03 (12:08)

Shows: `parentImage` `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, `parentProcessId` 7828, `parentUser` DESKTOP-T6T1N2V\aigbo, `processId` 9108, product Internet Explorer.
Demonstrates: Parent-child relationship: PowerShell launched `mshta.exe`, the pattern the rules describe.
Limitations: Photograph at an angle.

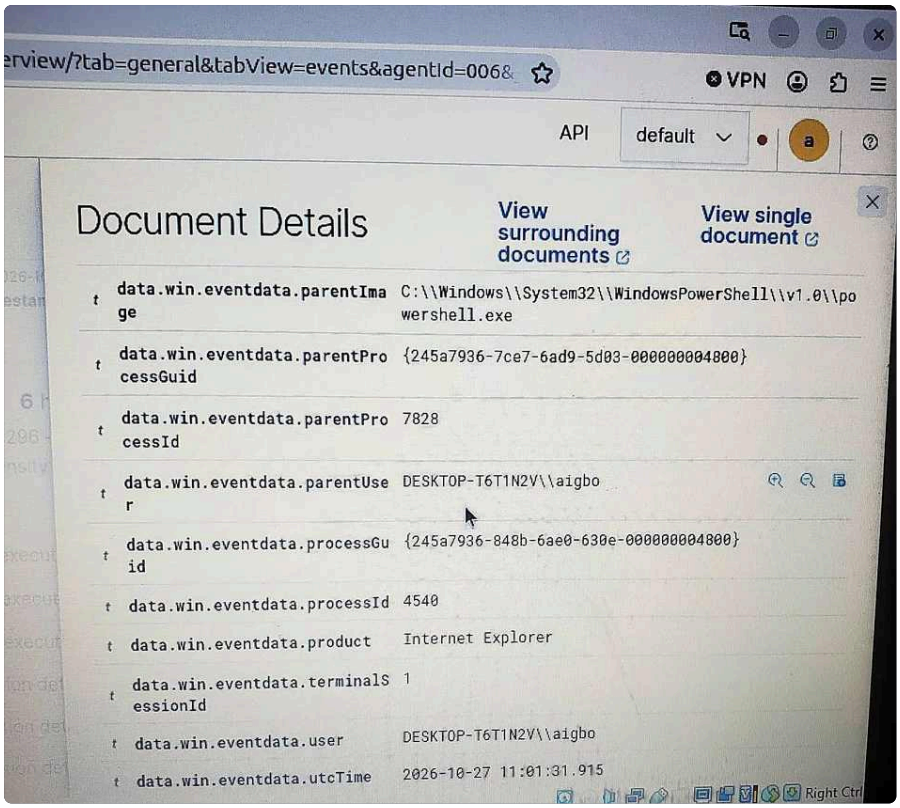


Figure 25. Alert details (4): Sysmon provider and raw message

Step: 10 **Source:** 20541937 (12:08)

Shows: `data.win.system` fields: channel `Microsoft-Windows-Sysmon/Operational`, computer `DESKTOP-T6T1N2V.HopeLabs.Local`, eventID 1, provider Microsoft-Windows-Sysmon, severityValue INFORMATION, and the "Process Create" message text with UtcTime 2026-10-27 11:00:14.023.
Demonstrates: The event provenance: Sysmon Event ID 1 on the endpoint, one hour behind the dashboard's local time.
Limitations: Photograph at an angle.

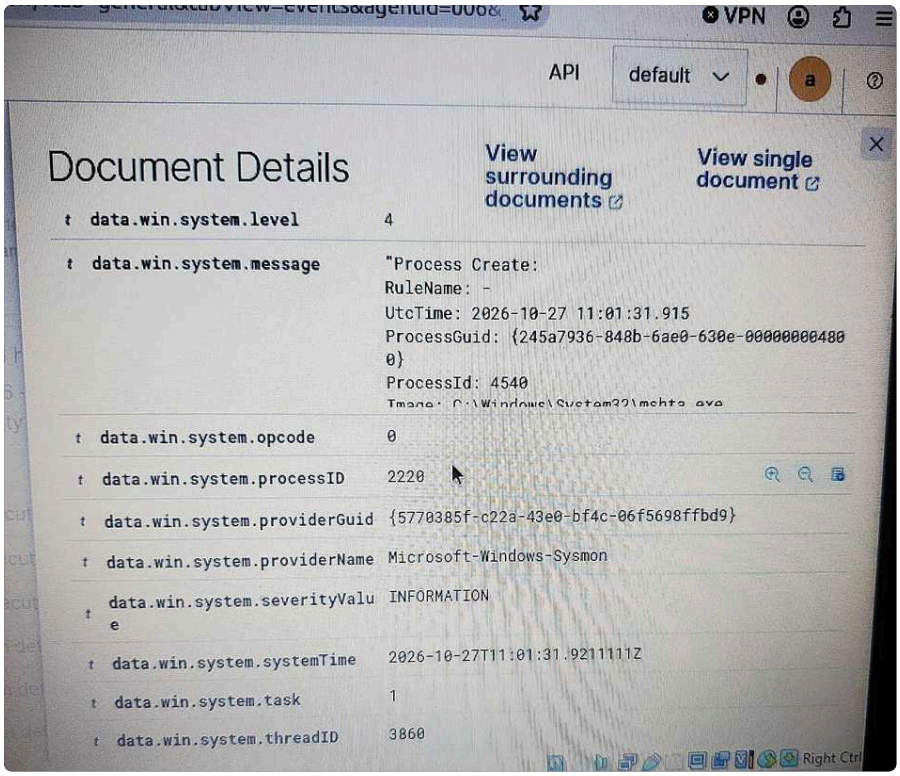


Figure 26. Alert details (5): decoder and full log

Step: 10 **Source:** 1594f9d1 (12:09)

Shows: `decoder.name` `windows_eventchannel`, `location` EventChannel, `manager.name` Ubuntu, and the beginning of `full_log`, which holds the original JSON-encoded Sysmon event. The rule description shown is "Multiple phishing execution indicators correlated", the 100107 alert.
Demonstrates: Decoding stage confirmed, and the correlation alert wraps the underlying event.
Limitations: Photograph at an angle.

CHAPTER 7

Configuration and Detection Logic

7.1 The Wazuh rules (reconstructed from the screenshots)

Evidence gap. The full `local_rules.xml` was never exported as a file, so the rules below are transcribed from the photographed nano screens and the `grep` output after repair. They are reconstructed, not copied. Verify them against your own file before reuse. Rules 100100, 100101 and earlier IDs also exist in the file and are not reproduced here.

```
<group name="phishing,office_exploit,">
  <!-- Office app spawning a shell or scripting engine - classic phishing payload trigger -->
  <rule id="100104" level="14">
    <if_sid>61603</if_sid>                          <!-- added during repair -->
    <field name="win.system.eventID">1</field>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)
(WINWORD|EXCEL|POWERPNT|OUTLOOK|MSACCESS)\.EXE</field>
    <field name="win.eventdata.image" type="pcre2">(?i)
(cmd|powershell|pwsh|wscript|cscript|mshta|rundll32|regsvr32|certutil)\.exe</field>
    <description>Possible Phishing Payload: Office application spawned a shell/script
interpreter</description>
    <mitre><id>T1566.001</id><id>T1204.002</id></mitre>
  </rule>

  <rule id="100105" level="13">
    <if_sid>61603</if_sid>
    <field name="win.system.eventID">1</field>
    <field name="win.eventdata.image" type="pcre2">(?i)mshta\.exe</field>
    <description>mshta.exe execution detected __ possible malicious HTA Phishing payload</description>
    <mitre><id>T1218.005</id></mitre>
  </rule>

  <rule id="100106" level="12">
    <if_sid>61603</if_sid>
    <field name="win.system.eventID">1</field>
    <field name="win.eventdata.image" type="pcre2">(?i)
(cmd|powershell|pwsh|wscript|cscript|mshta)\.exe</field>
    <field name="win.eventdata.parentCommandLine" type="pcre2">(?i)
(\\Downloads\\|\\AppData\\Local\\Temp\\|\\Users\\Public\\)</field>
    <description>Script/shell execution launched from a common phishing drop path
(Downloads/Temp/public)</description>
    <mitre><id>T1566</id></mitre>
  </rule>

  <rule id="100107" level="15" frequency="2" timeframe="180">
    <if_matched_sid>100105</if_matched_sid>        <!-- corrected during repair -->
    <description>Multiple phishing execution indicators correlated __ likely active payload
chain</description>
  </rule>
</group>

<group name="windows,sysmon,powershell,">
  <rule id="100510" level="5">
    <if_sid>61603</if_sid>
    <field name="win.system.channel">Microsoft-Windows-Sysmon/Operational</field>
    <field name="win.system.eventID">1</field>
    <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
    <description>Lab detection: PowerShell process creation observed by Sysmon</description>
    <mitre><id>T1059.001</id></mitre>
  </rule>
</group>

<group name="gophish_lab,phishing_simulation,">
  <rule id="100110" level="10">
    <if_sid>61603</if_sid>
    <field name="win.system.eventID">1</field>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)
(chrome|msedge|firefox|iexplore)\.exe$</field>
    <field name="win.eventdata.image" type="pcre2">(?i)
(cmd|powershell|pwsh|wscript|cscript|mshta)\.exe$</field>
    <description>GoPhish lab: Browser spawned a suspicious command interpreter.</description>
    <mitre><id>T1204.001</id><id>T1059</id></mitre>
  </rule>
```

```
  <rule id="100111" level="12">
    <if_sid>61603</if_sid>
    <field name="win.system.eventID">1</field>
    <field name="win.eventdata.image" type="pcre2">(?i)
(cmd|powershell|pwsh|wscript|cscript|mshta)\.exe</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)
(Downloads|AppData\\Local\\Temp|Users\\Public)</field>
    <description>Script/shell execution running from a common phishing drop path (current directory)
</description>
    <mitre><id>T1566</id></mitre>
  </rule>
</group>
```

7.2 Line-by-line explanation

Table 9. Rule elements explained

Element	What it does	Original / changed / recommended
`<rule id level>`	Unique custom ID (100000+) and severity 0–15. Levels in the lab: 5 (PowerShell launch), 10, 12, 13, 14, 15.	Original
`<if_sid>61603</if_sid>`	Makes the rule a child of the built-in Sysmon Event 1 rule, so it is evaluated for that event type.	**Changed**: added to 100104, 100105, 100106, 100110, 100111, 100510
`win.system.eventID = 1`	Only process-creation events.	Original; redundant once `if_sid` is present, harmless
`win.eventdata.image` (pcre2)	Path of the program that started. `(?i)` is case-insensitive; `\.exe` escapes the dot; `$` anchors the end of the string.	Original
`win.eventdata.commandLine` (100111)	The program's own command line, which includes `Downloads` when run from that folder.	Original; this is why the test fires 100111
`win.eventdata.parentCommandLine` (100106)	The *parent's* command line, not the child's. Tests in this lab did not satisfy it.	Original; recommend changing to `commandLine` or `currentDirectory`
`frequency="2" timeframe="180"`	Correlation: fires on the 2nd matching event within 180 seconds.	Original values
`<if_matched_sid>`	Names the rule whose matches are counted. It accepts one rule ID, not a comma-separated list.	**Changed**: 100107 now counts 100105
`<mitre>`	ATT&CK technique IDs attached to the alert.	Original

7.3 Rule inventory and status

Table 10. Custom rules and verification status

Rule	Level	Detects	Parent / counted rule	Status
100104	14	Office app spawning a shell or script host	if_sid 61603	Defined, **not tested**
100105	13	`mshta.exe` execution (T1218.005)	if_sid 61603	**Verified**
100106	12	Shell or script host whose parent command line is in a drop path	if_sid 61603	Defined, **not tested**
100107	15	Two 100105 matches within 180 s	if_matched_sid 100105	**Verified** (3 alerts)
100110	10	Browser parent launching a command interpreter	if_sid 61603	Defined, **not tested** (needs GoPhish or a browser-launched shell)
100111	12	Shell or script host with Downloads/Temp/Public in its command line	if_sid 61603	**Verified**
100510	5	Any PowerShell process creation (T1059.001)	if_sid 61603	**Verified** (5 alerts)

7.4 Equivalent Splunk searches

Splunk receives the same Sysmon events in classic Windows event format (fields `EventCode`, `Image`, `ParentImage`, `CommandLine`, `User`, `CurrentDirectory`, `IntegrityLevel`, `Hashes`), index `sysmon`. Searches marked **run** appear in the screenshots; the others were written but their results were not captured.

Table 11. Wazuh rules and their Splunk counterparts

Wazuh rule	SPL search	Status
100105 (mshta)	`index=sysmon EventCode=1 Image="*\\mshta.exe" | table _time host EventCode User ParentImage Image CommandLine`	**Run** (Figures 27, 29)
100510 (PowerShell)	`index=sysmon EventCode=1 Image="*\\powershell.exe" | stats count by host User ParentImage CommandLine`	**Run** (Figure 30)
100107 (correlation)	`index=sysmon EventCode=1 Image="*\\mshta.exe" | sort 0 _time | streamstats time_window=3m count as hits by host | where hits>=2 | table _time host User ParentImage CommandLine hits`	**Run** with `hits>2` (Figure 33)
100111 (drop path)	`index=sysmon EventCode=1 | regex Image="(?i)\\\\(cmd|powershell|pwsh|wscript|cscript|mshta)\.exe$" | regex CommandLine="(?i)(Downloads|AppData\\\\Local\\\\Temp|Users\\\\Public)"`	Written, **not evidenced**
100110 (browser parent)	`index=sysmon EventCode=1 ParentImage IN ("*\\chrome.exe","*\\msedge.exe","*\\firefox.exe","*\\iexplore.exe") Image IN ("*\\cmd.exe","*\\powershell.exe","*\\wscript.exe","*\\mshta.exe")`	Written, **not evidenced**

> **Note.** `streamstats time_window=3m` counts matching events in a sliding three-minute window, which avoids the boundary problem of a fixed `bin` time bucket. The lab run used `where hits>2`, which requires three events; use `hits>=2` to match Wazuh's frequency of 2.

CHAPTER 8

Testing and Results

8.1 Test matrix

Table 12. Test scenarios and verification status

Test	Command / action	Expected	Observed	Status
T1 Sysmon logging	`Get-WinEvent ... Id=1`	Event 1 for test commands	Events at 10:43–10:47	**Pass** (Fig. 7)
T2 Rules load	`wazuh-analysisd -t | grep -i 1001`	No lines	Initially warnings; empty after repairs	**Pass** after repair (Fig. 11)
T3 Drop-path rule	`powershell.exe -Command "cd ...\Downloads; Write-Host 'Lab test v8'"`	Rule 100111 fires	Alert level 12 at 11:26	**Pass** (Fig. 12)
T4 mshta rule	`mshta.exe "javascript:closed()"`	Rule 100105 fires	100105 alerts in file and dashboard	**Pass** (Fig. 20, 22)
T5 PowerShell rule	`powershell.exe -Command whoami`	Rule 100510 fires	5 alerts counted	**Pass** (Fig. 16)
T6 Correlation	Two `mshta.exe` runs within seconds	Rule 100107 fires	3 alerts (12:00:24, 12:00:55, 12:01:40)	**Pass** (Fig. 19, 20)
T7 Office parent	Office app spawning a shell	Rule 100104 fires	Not attempted	**Untested**
T8 Parent command line	Shell whose parent command line is in a drop path	Rule 100106 fires	Not attempted; ordinary tests do not satisfy it	**Untested**
T9 Browser parent	Browser launching a shell	Rule 100110 fires	Not attempted	**Untested**
T10 Splunk ingestion	`index=sysmon EventCode=1`	Events present	1,197 events in 24 h	**Pass** (Fig. 31)
T11 Splunk mshta search	SPL for `mshta.exe`	Test runs listed	10 events	**Pass** (Fig. 27, 29)
T12 Splunk correlation	`streamstats` search	Bursts flagged	2 result events (with `hits>2`)	**Pass** (Fig. 33)
T13 Splunk saved alerts	Save searches as scheduled alerts	Triggered Alerts entries	No evidence captured	**Not evidenced**

8.2 Cross-check between Wazuh and Splunk

Two independent systems recorded the same events. The Splunk `mshta.exe` search lists ten process-creation events; the Wazuh dashboard, filtered to the `mshta.exe` command line, shows six alerts (3 × 100105 and 3 × 100107). The table lines them up.

Table 13. mshta.exe executions seen by Splunk and by Wazuh (times are one second apart because alerts are written after analysis)

Event time (WAT)	In Splunk	Wazuh alert	Interpretation
08:56:39	Yes (Fig. 29)	None in dashboard filter	Before the fix: rules not firing, consistent with the root cause
10:42:42	Yes (Fig. 32, 29)	None	Before the fix
11:44:39	Yes	100105 at 11:44:40	First verified detection of mshta
11:48:40	Yes	100105 at 11:48:41	Verified
12:00:14	Yes	100105 at 12:00:15	Verified
12:00:23, 12:00:53, 12:01:31	Yes (three events)	100107 at 12:00:24, 12:00:55, 12:01:40	Second and later matches in 180 s raise the correlation alert
14:51:30, 14:53:20	Yes (Fig. 29)	Not captured	After the Wazuh dashboard screenshots (12:07); Wazuh status for these is unknown

Verified. Splunk's ten events and Wazuh's six alerts agree for the period both screenshots cover. The first two Splunk events predate the repair, which supports the explanation that the custom rules were not evaluated before `<if_sid>` was added. Wazuh's absence of alerts for those two is read from a filtered dashboard view, so it is consistent with, not proof of, the cause.

8.3 What failed, what is unverified

- Failed first, then fixed: all custom rules (no alerts), rule 100107 (invalid reference), rule 100101 (accidentally changed, then restored).
- Unverified: rules 100104, 100106 and 100110; Splunk saved alerts; the precise internal reason a top-level rule is not evaluated; behaviour under a realistic phishing delivery.
- Noise: PowerShell launches by the Wazuh agent itself (`secedit` collection scripts) and by CompatTelRunner would also trigger rule 100510 (Figure 30).

CHAPTER 9

Evidence and Screenshot Analysis

The figures appear beside the procedure steps they support (Chapter 6). This chapter adds the Splunk evidence, which supports the Splunk searches in Chapter 7 and the cross-check in Chapter 8, and describes how the images were prepared.

9.1 Image preparation and evidence integrity

- Original photographs and screenshots are retained unchanged.
- Report copies are cropped to the relevant screen area, contrast-adjusted and sharpened. No text, value, timestamp, IP address, rule ID or error message was edited, redrawn, added or removed.
- No annotations were added, so every pixel shown is original evidence.
- Most Wazuh and terminal images are phone photographs of a monitor. They show perspective distortion and glare; where text was too small to read, the report relies on typed command output or other figures instead of guessing.
- Duplicate screenshots were removed (for example, three byte-identical Splunk captures of the same results page). No redaction was needed: the images show private lab addresses, lab usernames and file hashes of a standard Windows binary, with no passwords or tokens.

9.2 Splunk evidence

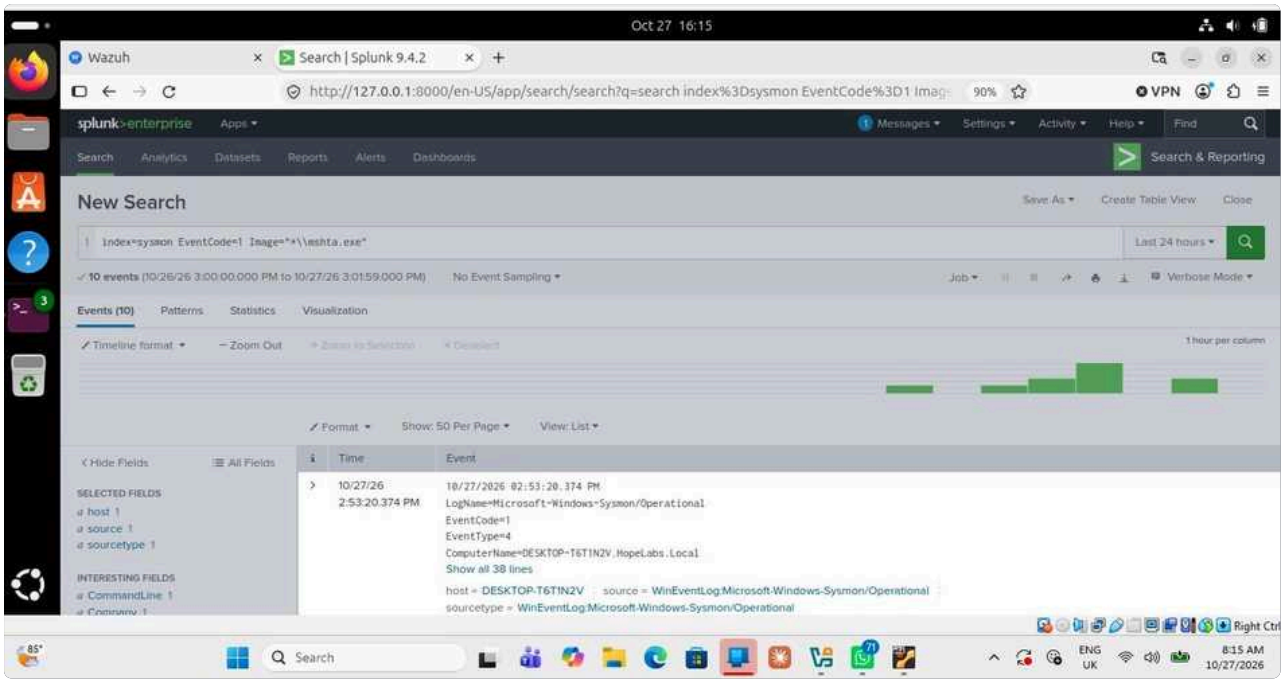


Figure 27. Splunk search for mshta.exe: 10 events over 24 hours

Step: Chapter 7 **Source:** 40339222 (Oct 27, 16:15)

Shows: Search `index=sysmon EventCode=1 Image="*\\mshta.exe"`, last 24 hours, 10 events. The histogram shows clusters of activity; the first event details show `EventCode=1`, computer `DESKTOP-T6T1N2V.HopeLabs.Local` and the Sysmon "Process Create" message.
Demonstrates: Splunk is ingesting Sysmon Event 1 from the endpoint and the `mshta.exe` search isolates the test runs.
Limitations: Screen capture of the Splunk web interface (browser zoom 90%).

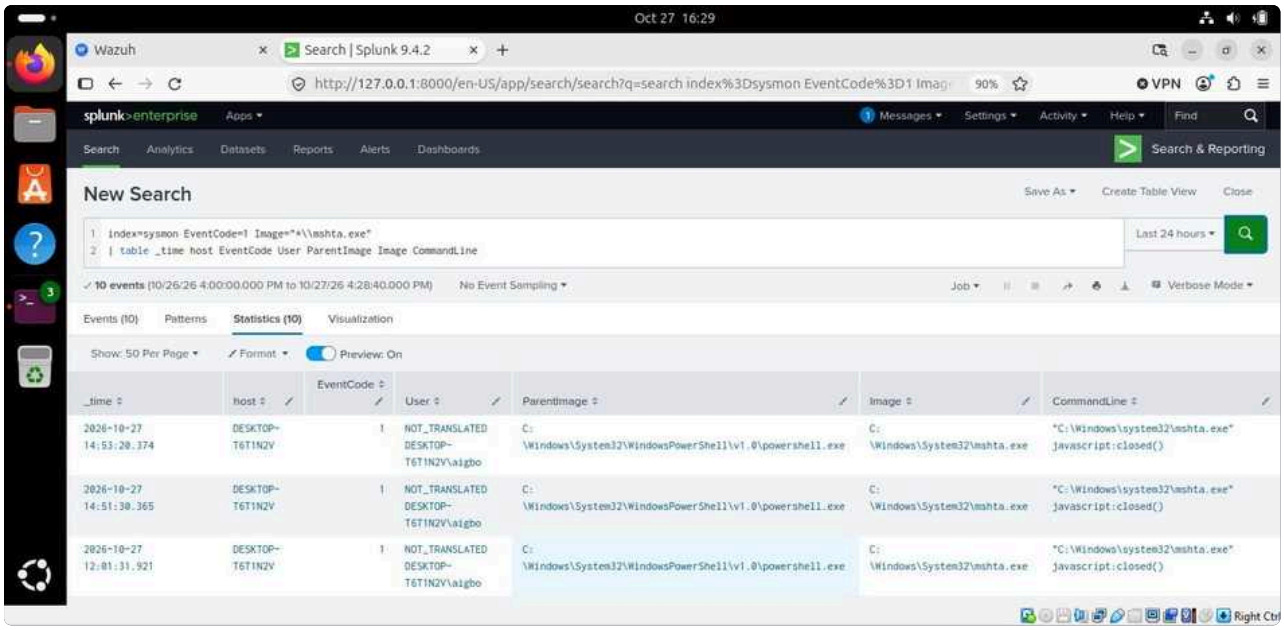


Figure 28. mshta.exe results as a table

Step: Chapter 7 **Source:** a172db43 (Oct 27, 16:29)

Shows: Statistics view: 14:53:20, 14:51:30 and 12:01:31 events, host DESKTOP-T6T1N2V, user `DESKTOP-T6T1N2V\aigbo` (displayed as `NOT_TRANSLATED` plus the user name), parent `powershell.exe`, image `mshta.exe`, command line `"C:\Windows\system32\mshta.exe" javascript:closed()`.

Demonstrates: The table form matches the Wazuh alert fields and shows the same parent-child pair.

Limitations: Only three of the ten rows are visible in this capture.

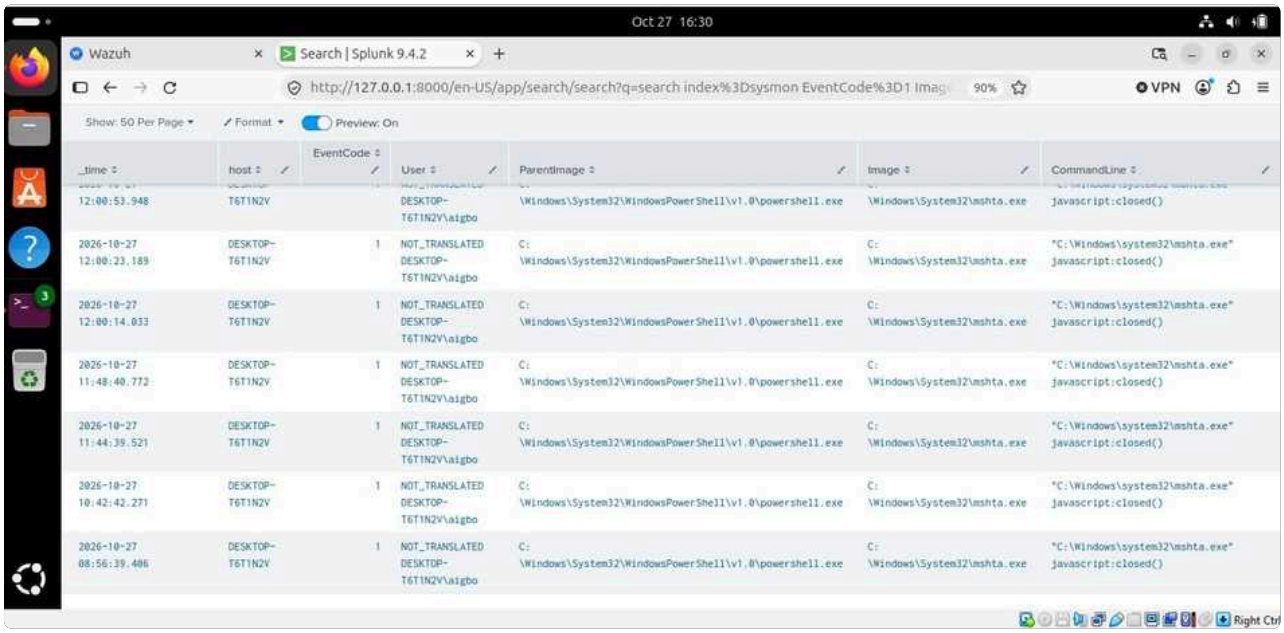


Figure 29. mshta.exe execution times in Splunk

Step: Chapter 8 **Source:** 37b08bdf (Oct 27, 16:30)

Shows: Rows for 12:00:53.948, 12:00:23.189, 12:00:14.033, 11:48:40.772, 11:44:39.521, 10:42:42.271 and 08:56:39.406 on 2026-10-27, all with parent `powershell.exe` and the same command line.

Demonstrates: These timestamps line up with the Wazuh alerts used in the cross-check table (Chapter 8).

Limitations: Image is partially cropped at the top by the table scroll position.

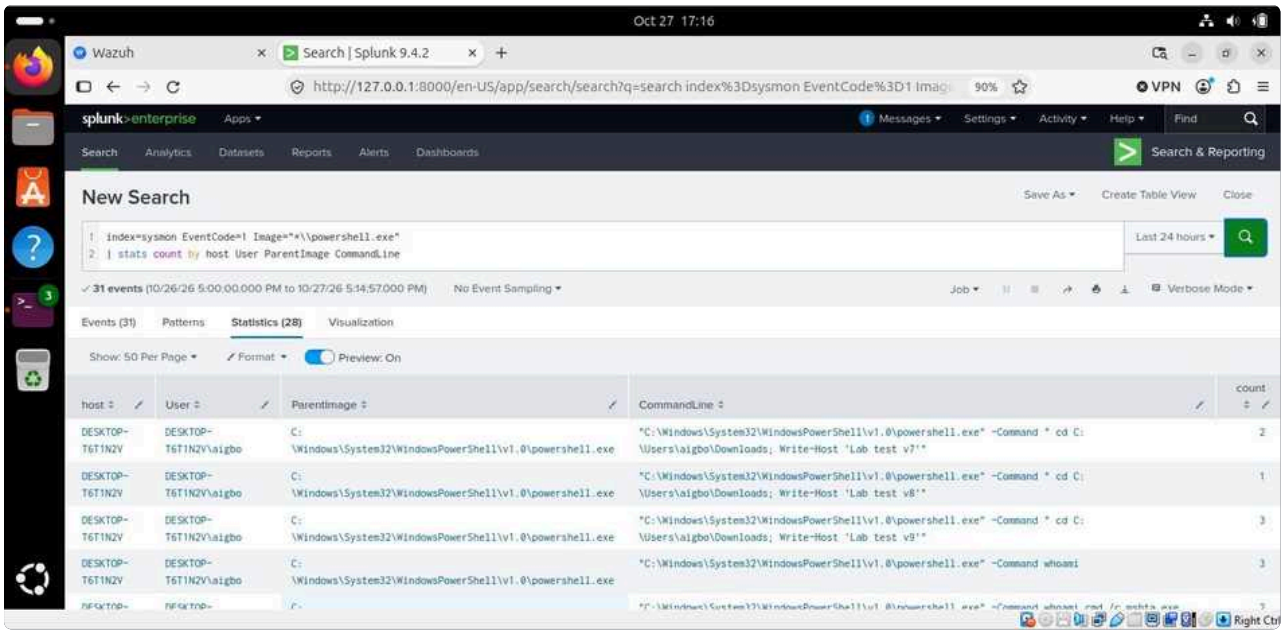


Figure 30. PowerShell launches grouped by command line

Step: Chapter 7 **Source:** d066f32a (Oct 27, 17:16)

Shows: `index=sysmon EventCode=1 Image="*\\powershell.exe" | stats count by host User ParentImage CommandLine` : 31 events, 28 result rows, including `Lab test v7` , `v8` , `v9` , `whoami` and many `secedit` commands started by `wazuh-agent.exe` .

Demonstrates: Shows both the test activity and legitimate noise from the Wazuh agent and Windows telemetry, relevant to false positives for rule 100510.

Limitations: Only the top rows are visible.

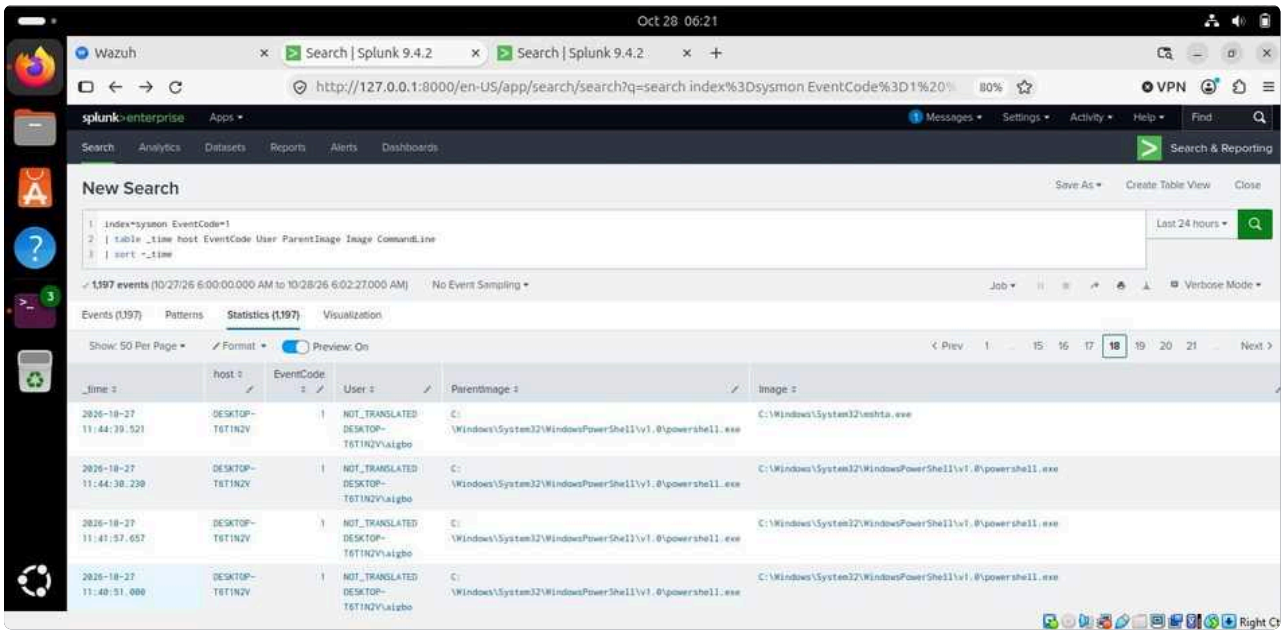


Figure 31. All Sysmon Event 1 records: 1,197 in 24 hours

Step: Chapter 7 **Source:** 810cb846 (Oct 28, 06:21)

Shows: `index=sysmon EventCode=1 | table _time host EventCode User ParentImage Image CommandLine | sort -_time` : 1,197 events across 24 hours, shown in a paged table.
Demonstrates: Ingestion volume is manageable, and a general search can see every process creation, not just detections.
Limitations: Photographed table page is a fragment.

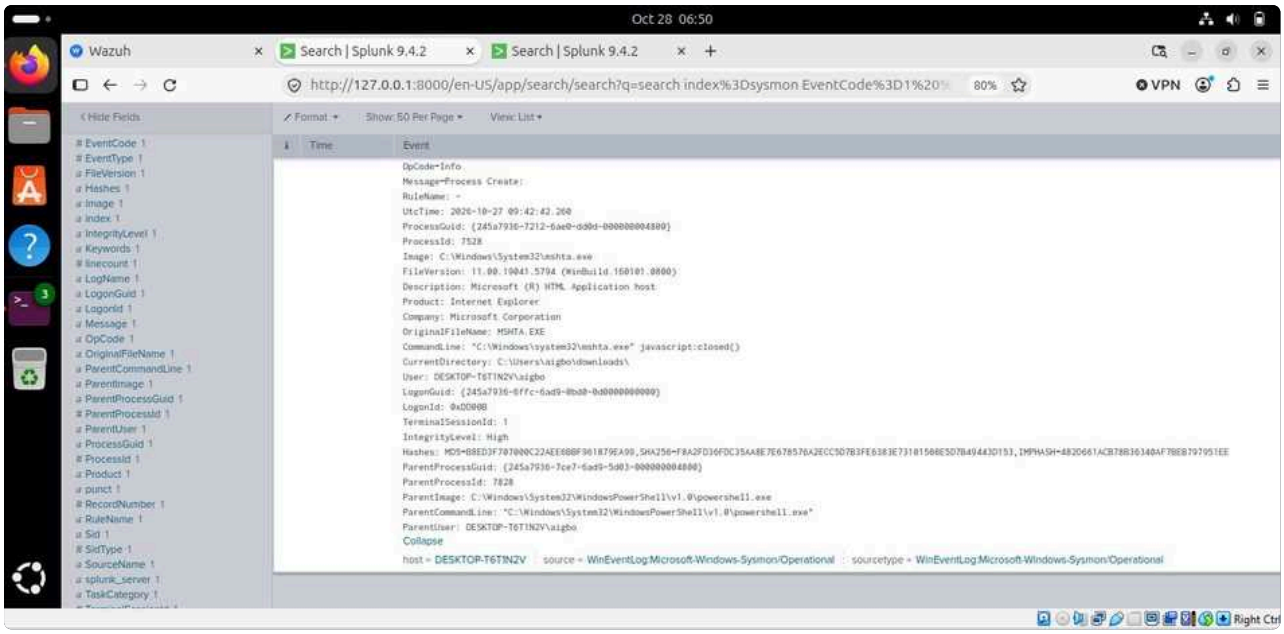


Figure 32. Full Sysmon event in Splunk

Step: Chapter 8 **Source:** 330b6dba (Oct 28, 06:50)

Shows: Expanded event from 10:42:42 on 27 Oct: UtcTime 2026-10-27 09:42:42.260, `ProcessId` 7528, `Image` `C:\Windows\System32\mshta.exe`, `CommandLine` `javascript:closed()`, `CurrentDirectory` `C:\Users\aigbo\downloads\`, integrity level High, hashes, parent `powershell.exe` (process 7828), parent user `DESKTOP-T6T1N2V\aigbo`.

Demonstrates: The same event provides every field needed for the Wazuh and Splunk detections. This test (10:42) occurred before the Wazuh repair.

Limitations: Hash values are not transcribed in the report text.

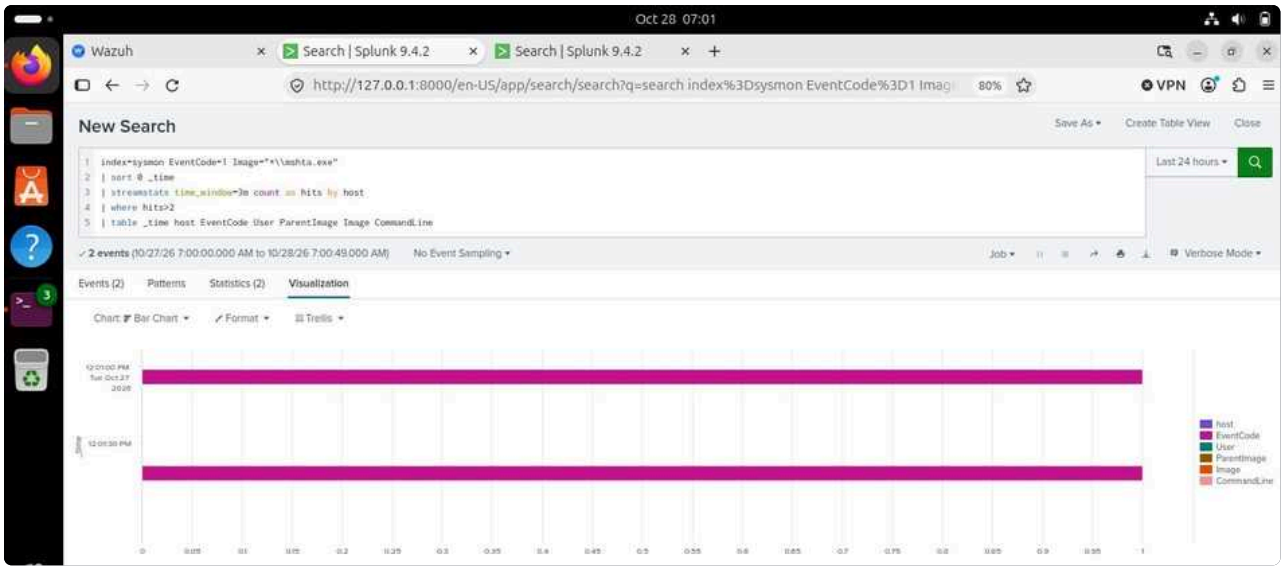


Figure 33. Correlation search result in Splunk

Step: Chapter 7 **Source:** 63f83b28 (Oct 28, 07:01)

Shows: SPL: `index=sysmon EventCode=1 Image="*\\mshta.exe" | sort 0 _time | streamstats time_window=3m count as hits by host | where hits>2 | table ...`. 2 events returned, last 24 hours. The view is on the Visualization tab, which draws the field names as bars; the Statistics tab holds the rows.
Demonstrates: The Splunk equivalent of rule 100107 flags repeated `mshta.exe` execution within three minutes (two events had three or more runs in the window).
Limitations: The chart is a default visualization of a table; it carries no information beyond a count of two result rows.

CHAPTER 10

Troubleshooting and Root-Cause Analysis

10.1 Problem 1: custom rules produced no alerts

Table 14. Problem 1 analysis

Item	Detail
Symptom	Custom rules never appeared in the dashboard after test commands. The dashboard showed built-in rule 92217 alerts only (Figure 8).
Possible causes considered	(a) Rules not reloaded; (b) syntax error; (c) Sysmon not logging Event 1; (d) Sysmon channel not collected by the agent; (e) rules without a parent rule.
Diagnostics	Event ID 1 present on Windows (Figure 7) excluded (c); built-in Sysmon alerts arriving excluded (d); `wazuh-analysisd -t` and restart addressed (a), (b); remaining difference was the missing parent (e).
Confirmed root cause	Rules had no `<if_sid>`. After adding `<if_sid>61603</if_sid>`, rule 100111 fired (Figure 12).
Uncertainty	The mechanism was not traced with `wazuh-logtest -v`; the explanation matches Wazuh's rule-tree design and the experimental result.

10.2 Problem 2: correlation rule 100107 rejected

The first correlation fix used a comma-separated list, `<if_matched_sid>100104,100105,100106,100111</if_matched_sid>`. `wazuh-analysisd -t` answered `WARNING: (7615): Invalid 'if_matched_sid' value: '100104,100105,100106,100111'. Rule '100107' will be ignored.` The element accepts one rule ID.

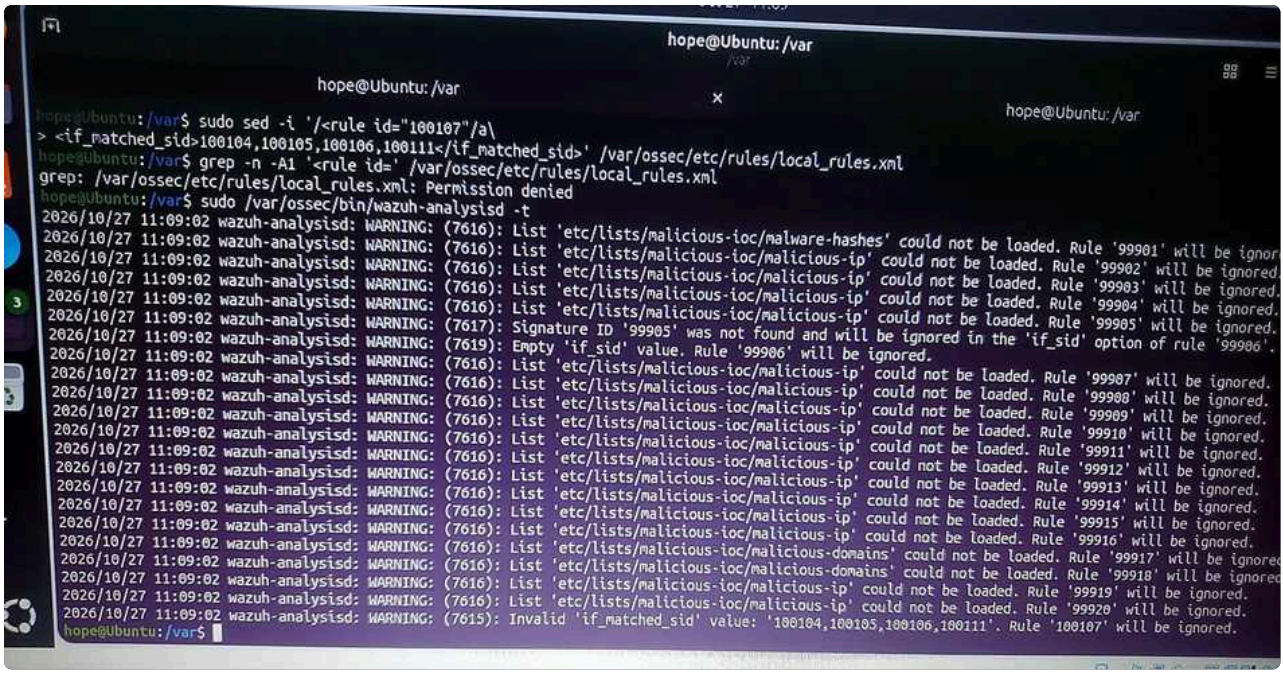


Figure 34. Invalid if_matched_sid reported by the rules test

Step: 10 **Source:** 9ce01652 (11:09)

Shows: The terminal shows a `sed` command that was denied ("Permission denied" because `grep` was run without `sudo`), then `wazuh-analysisd -t` output ending with the (7615) warning that rule 100107 is ignored. The many (7616) warnings above it concern missing `malicious-ioc` list files.
Demonstrates: A rule with an invalid reference is silently disabled at load time, so a rules test after every edit is essential.
Limitations: The IOC list warnings (rules 99901–99920) are from a separate ruleset and are unrelated to this lab.

10.3 Problem 3: editing mistakes on the command line

Two sequential command-line errors are documented here. The second caused unintended damage that the backup then repaired.

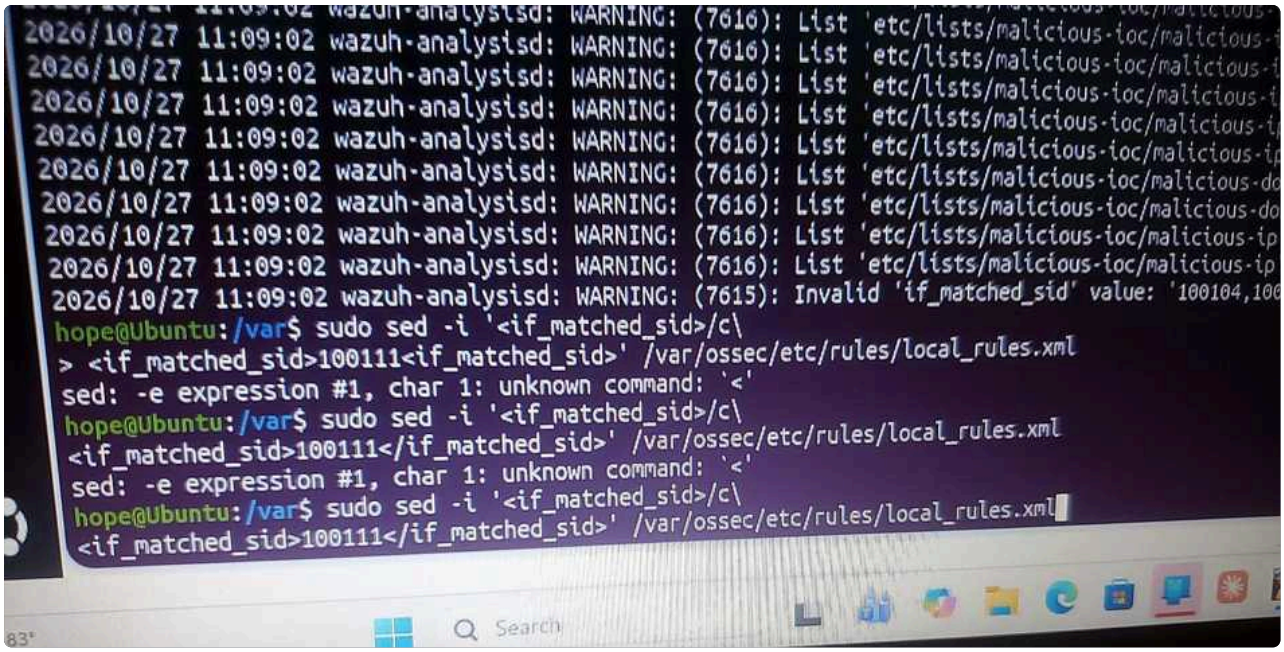


Figure 35. sed syntax error: unknown command '<'

Step: 10 **Source:** 31f60ded (11:13)

Shows: The `sed` command with `c\` was split across lines when pasted, so the pattern lost its leading slash. `sed` answered "unknown command: `<`" three times.
Demonstrates: Multi-line `sed` commands are fragile when pasted. Single-line commands or a text editor are safer.
Limitations: Photograph; upper lines are IOC warnings from the previous command.

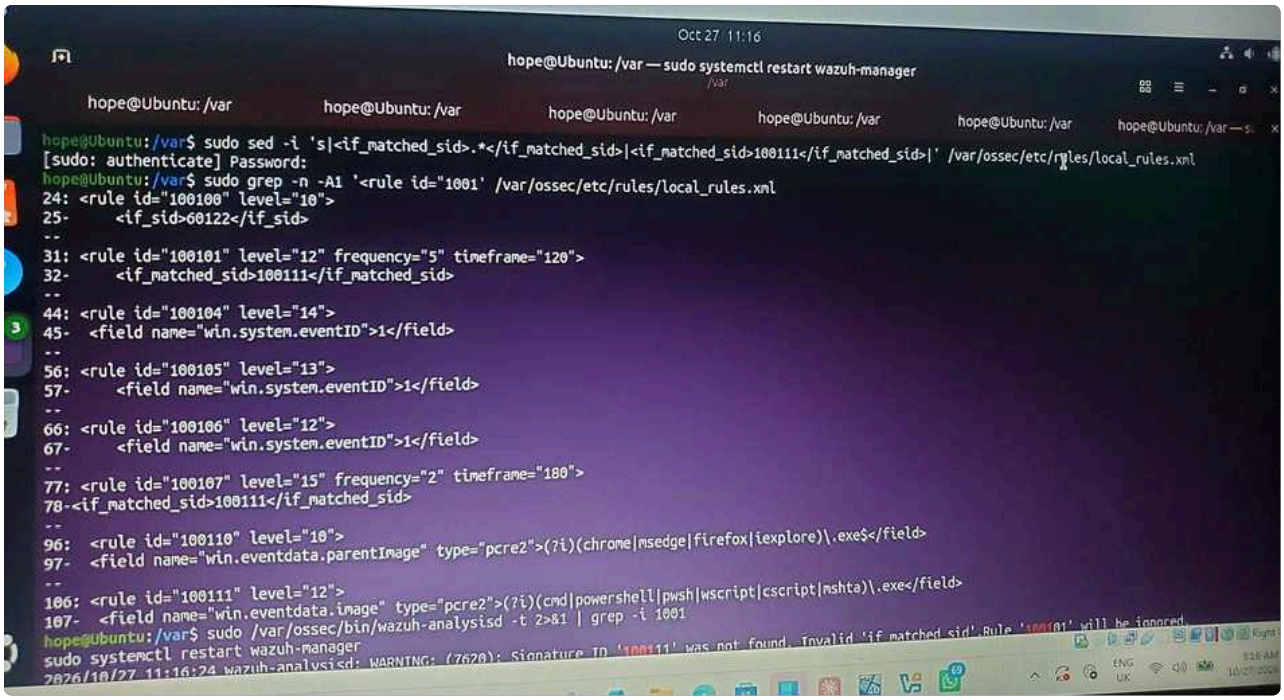


Figure 36. Over-broad sed replaced every if_matched_sid

Step: 10 **Source:** e16520fb (11:16)

Shows: The substitution `s|<if_matched_sid>.*</if_matched_sid>|<if_matched_sid>100111</if_matched_sid>|` changed *every* such line in the file. Line 32 (rule 100101) now points at 100111 as well as line 78 (rule 100107). The `grep` also shows that no `<if_sid>` lines yet exist under 100104–100106.
Demonstrates: A global substitution altered a rule that was not meant to change. The backup showed the original value of rule 100101 (`100100`).
Limitations: Rule 100101 and the older rules are not part of this report's detections.

Corrective action: restore rule 100101 to `100100` with a range-limited substitution (Step 6), then re-run the rules test. **Prevention:** always scope `sed` to a single rule with an address range or edit in nano; compare with `diff local_rules.xml local_rules.xml.bak` after each change.

10.4 Problem 4: forward references

Rule 100101 and rule 100107 referenced rule 100111, which is defined later in the file. Wazuh reported `Signature ID '100111' was not found` because a rule must be defined before another rule refers to it. Rule 100101 was restored to its original reference and rule 100107 was pointed at rule 100105, which precedes it.

10.5 Problem 5: tests that never ran

- Two commands were pasted on one line (`"whoami" cmd /c mshta.exe ...`) and PowerShell raised parser errors (Figure 13). No process was created, so no alert was possible.
- `mshta.exe javascript:close()` typed without quotes causes "An expression was expected after '('" because PowerShell parses the parentheses. Use quotes or `cmd /c` .
- The `uniq -c` check looked at only the last 200 lines. Rule 100510 fired earlier and was found with `grep -c` over the whole file (Figure 16).

10.6 Unrelated noise

- The (7616) and (7617)/(7619) warnings about `malicious-ioc` lists and rules 99901–99920 belong to an unrelated IOC ruleset with missing list files. They were ignored and do not affect the custom rules.
- Built-in rule 92213 (level 15, "Executable file dropped in folder commonly used by malware") fired alongside rule 100111 because PowerShell writes its own temporary policy-test script (`__PSScriptPolicyTest_*.ps1`) to the Temp folder. It is a benign side effect of running PowerShell.

CHAPTER 11

Security Findings and Risk Analysis

These findings concern the lab's detection capability. The activity itself was a harmless simulation, not an incident.

Table 15. Findings and confidence

#	Finding	Type	Evidence	Likely impact
F1	Custom detection rules were deployed but silently inactive until a parent rule was added.	**Confirmed**	Fig. 8, 12; Splunk 08:56 and 10:42 events had no Wazuh alerts	A deployed but inactive rule gives false assurance; real attacks would go unalerted.
F2	A rule referencing another rule with an invalid value is dropped at load time without breaking the service.	**Confirmed**	Fig. 34, 11	Correlation silently lost; needs an automated rules test after every change.
F3	`powershell.exe` launching `mshta.exe` at High integrity, from the Downloads folder, is detected as level 13 and, when repeated, level 15.	**Confirmed** (simulated)	Fig. 20–26	Matches a realistic post-phishing pattern; detection is useful.
F4	Rule 100510 is noisy: the Wazuh agent and Windows telemetry also launch PowerShell.	**Confirmed**	Fig. 30	False positives and alert fatigue at scale.
F5	Rule 100106 tests the parent command line, which many real chains will not satisfy.	Possible risk (design)	Rule text, Fig. 3	False negatives.
F6	Rules 100104, 100106 and 100110 are unverified.	Evidence gap	Section 8.1	Unknown behaviour in the cases they were written for.
F7	The lab ran an administrator shell, so every event carries High integrity; behaviour for a standard user may differ.	Assumption	Fig. 23	Detections may need level adjustments.

11.1 Risk assessment

The highest-value finding is F1. A rule that has never been seen to fire is an untested control. The lab's repair loop (write, test, verify in the raw alert file, verify in the dashboard, cross-check in a second tool) is itself the mitigation and should be repeated for every rule added.

CHAPTER 12

Mitigation and Remediation

Everything below except item M1 (parent rule) and the repairs in Chapter 6 is a **recommendation**. None was implemented in this lab.

Table 16. Mitigations

#	Recommendation	Objective and approach	Benefit / impact	Verify	Status
M1	Attach rules to the correct parent (`if_sid` 61603) and correct invalid references	Rules evaluate for Sysmon Event 1	Alerts fire	Test runs, Figs. 12, 20	**Implemented**
M2	Add a rules-test step to every change: `wazuh-analysisd -t` , then a synthetic test	Catch invalid rules before they are silently ignored	Fewer silent failures	Test mode shows no 1001xx lines	Recommended
M3	Narrow rule 100510: exclude parents `wazuh-agent.exe` and `CompatTelRunner.exe` , or alert only on encoded or suspicious command lines	Reduce noise	Less alert fatigue; risk of missing a spoofed parent	Splunk query in Fig. 30 minus those parents	Recommended
M4	Change rule 100106 to test `commandLine` or `currentDirectory`	Match the lab's actual drop-path behaviour	Fewer false negatives	Run a drop-path test	Recommended
M5	Block or restrict `mshta.exe` where HTA is not needed (AppLocker, Windows Defender Application Control, or an Attack Surface Reduction rule)	Prevent, not only detect, T1218.005	Removes the technique; may break legacy HTA apps	Attempt `mshta.exe` and confirm it is blocked and logged	Recommended
M6	Enable PowerShell Script Block Logging	Capture decoded script content	Deeper PowerShell visibility; more log volume	Event ID 4104 present	Recommended
M7	Back up rules and snapshot VMs after each verified milestone	Fast rollback	Recovery in minutes	Restore test	Recommended

CHAPTER 13

What I Learned

This section is written in the first person and reflects what actually happened in the lab.

Conceptual knowledge

I learned to think of detection as a chain with seven separate stages (Chapter 3) and to test them one at a time. When the dashboard showed nothing, the question was not "is Wazuh broken?" but "which stage failed?". Showing that Sysmon wrote Event ID 1 on Windows and that other Sysmon alerts reached the dashboard told me the fault lay in my own rules.

Practical technical knowledge

I learned that Wazuh custom rules for Sysmon events need a parent rule (`if_sid` 61603), that `if_matched_sid` takes a single ID, that a rule must be defined before another rule uses it, and that `wazuh-analysisd -t` exposes problems that a service restart hides. I learned to read Sysmon Event 1 fields (`Image`, `CommandLine`, `CurrentDirectory`, `ParentImage`) and why parent and child together are more telling than either alone.

Troubleshooting and care with commands

My command-line edits taught me to back up first, keep commands to a single line, scope `sed` to one rule, and to read the output after each change. An over-broad `sed` changed an unrelated rule; the backup showed me the original value. I also learned that a missing alert can mean the test never ran (quoting errors in PowerShell), and that a count of 0 can be a timing issue and not a broken rule.

Evidence analysis and detection engineering

Using two tools was valuable: Splunk's list of ten `mshta.exe` events and Wazuh's six alerts matched for the overlapping period, and the first two events with no alert showed what the missing parent had cost. I also learned to distinguish verified results (four rules fired) from written-but-untested rules (three) and to say so.

Areas for further development

- Run a real phishing simulation (GoPhish) and verify rule 100110 and Office-parent rule 100104.
- Use `wazuh-logtest -v` to trace rule evaluation instead of relying on result-based inference.
- Save Splunk searches as alerts and document the triggered alerts.
- Tune noisy rules and measure false-positive rates over a longer period.

CHAPTER 14

Practical Knowledge and Competencies

Table 17. Skills demonstrated, with the evidence for each

Skill	Task in this lab	Tool	Evidence	Level shown
Endpoint telemetry	Confirm Sysmon Event 1 on Windows	Sysmon, PowerShell `Get-WinEvent`	Fig. 7	Independent
Detection engineering	Write and repair custom Wazuh rules	`local_rules.xml`, nano, perl, sed	Figs. 2–5, 9, 10	Independent, with troubleshooting
SIEM validation	Verify alerts in raw file and dashboard	`alerts.json`, grep, Wazuh dashboard	Figs. 12, 16, 19, 20	Independent
Correlation logic	Frequency/timeframe rule 100107; Splunk `streamstats`	Wazuh, Splunk SPL	Figs. 17–19, 33	Independent
Threat mapping	ATT&CK T1218.005, T1059.001, T1566, T1204	Rule `<mitre>` tags	Fig. 21	Independent
Splunk searching	Search Sysmon events, stats, table	Splunk Enterprise	Figs. 27–33	Independent
Active simulation (phishing delivery)	GoPhish campaign	GoPhish	None	**Not demonstrated**
Alert management in Splunk	Scheduled alerts, Triggered Alerts	Splunk	None	**Not demonstrated**

14.1 Recommended next exercises

1. Add a GoPhish campaign and trigger rule 100110 with a browser-launched shell.
2. Write a rule for encoded PowerShell (`-enc` , `-EncodedCommand`) and validate it in Wazuh and Splunk.
3. Test rule 100104 with an Office document that launches a harmless `cmd.exe` echo.
4. Save the Splunk searches as alerts and document the triggered alerts.
5. Compare detection latency between the two systems.

CHAPTER 15

Reproduction Guide

This guide lets another learner recreate the lab. Use the checklist first, then the detailed parts that follow.

Table 18. Reproduction checklist

#	Do this	Where	Pass when	Detail
R1	Confirm Sysmon logs Event ID 1	Windows	`Get-WinEvent` lists Process Create events	Part A
R2	Confirm Wazuh receives Sysmon alerts from the agent	Wazuh dashboard	Any Sysmon-based alert for the agent appears	Part A
R3	Back up `local_rules.xml`	Ubuntu	`.bak` file exists	Part B
R4	Add the rules from Chapter 7 (or Appendix E file), each with `<if_sid>61603</if_sid>`	Ubuntu	File saved	Part B
R5	Run the rules test	Ubuntu	`wazuh-analysisd -t 2>&1 | grep -i 1001` prints nothing	Part B
R6	Restart the manager	Ubuntu	Status shows `active (running)`	Part B
R7	Run the three tests	Windows (admin)	Commands run with no red errors	Part C
R8	Count alerts	Ubuntu	100111, 100105, 100510 counts above 0; 100107 above 0 after two quick mshta runs	Part D
R9	View alerts in the dashboard	Wazuh dashboard	Filter shows the four rule IDs	Part D
R10	(Optional) Repeat the searches in Splunk	Splunk	`mshta.exe` search lists your test runs	Part E

Steps marked **(captured)** come from the evidence. Steps marked **(standard)** are normal procedure that was not captured in screenshots; check them against current vendor documentation.

15.1 Starting state

- Ubuntu VM with Wazuh manager, indexer and dashboard running; an administrator account.
- Windows 10 VM with Sysmon installed and logging Event ID 1, and the Wazuh agent enrolled and showing Active.
- Both VMs on a host-only network (this lab: 192.168.56.0/24).
- Optional: Splunk Enterprise on the Ubuntu VM and a method to forward Windows events to it.

15.2 Part A: confirm the telemetry

1. On Windows, run `Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=1} -MaxEvents 5`. You should see Process Create events. **(captured)**
2. In the Wazuh dashboard, open Threat Hunting, choose the agent and Events; confirm other Sysmon-based alerts (for example rule 92217) arrive. **(captured)**
3. If nothing arrives, check that the agent's `ossec.conf` has a `<localfile>` for `Microsoft-Windows-Sysmon/Operational` with `log_format eventchannel` and restart the agent. **(standard)**

15.3 Part B: install the rules

1. Back up: `sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak` **(captured)**.
2. Open the file: `sudo nano /var/ossec/etc/rules/local_rules.xml` **(captured)**. Paste the rules from Chapter 7 inside the existing structure, keeping the `<if_sid>61603</if_sid>` lines.
3. Validate: `sudo /var/ossec/bin/wazuh-analysisd -t 2>&1 | grep -i 1001`. Expect no output. **(captured)**
4. Restart: `sudo systemctl restart wazuh-manager`; confirm `sudo systemctl status wazuh-manager` shows active (running). **(captured)**

> **Warning.** Define each rule before any rule that refers to it. If 1001xx warnings appear, fix them before testing. Use single-line `sed` commands scoped to one rule, or nano; do not use unscoped substitutions.

15.4 Part C: run the tests (Windows, administrator PowerShell)

```
powershell.exe -Command "cd $env:USERPROFILE\Downloads; Write-Host 'Lab test v8'"   # expect rule
100111
powershell.exe -Command whoami                                                      # expect rule
100510
cmd /c mshta.exe "javascript:close()"                                               # expect rule
100105; run twice within 3 minutes for 100107
```

Enter each command on its own line and press Enter after each. Wait about 30 seconds between tests.

15.5 Part D: verify

```
sudo grep -c '"id":"100510"' /var/ossec/logs/alerts/alerts.json
sudo grep -c '"id":"100107"' /var/ossec/logs/alerts/alerts.json
sudo grep -oE '"id":"1001[0-9]+"' /var/ossec/logs/alerts/alerts.json | sort | uniq -c
```

Expected: each of 100111, 100105 and 100510 gives a count above 0 (the lab saw 5 for 100510); 100107 gives a count above 0 only after two `mshta.exe` runs less than 180 seconds apart (the lab saw 3). Counts accumulate across runs, so your numbers will differ.

In the dashboard, filter `rule.id: (100105 OR 100107 OR 100111 OR 100510)`, set the time range to cover your tests, and add columns `rule.id`, `rule.level`, `rule.description` and `data.win.eventdata.commandLine`. **(captured)**

15.6 Part E: Splunk (optional)

1. Create an index named `sysmon`. **(standard)**
2. Forward the Sysmon operational channel (classic Windows event format) to that index; the lab's events carry sourcetype `WinEventLog:Microsoft-Windows-Sysmon/Operational`. The exact forwarder configuration used in this lab was not captured. **(standard)**
3. Run the searches in Chapter 7. **(captured for mshta, PowerShell, correlation)**
4. Save each search as a scheduled alert (every 5 minutes, trigger when results > 0) and trigger it with the test commands. **(standard; not evidenced in this lab)**

15.7 Troubleshooting

Symptom	Likely cause	Fix
No alert after the test	Test command did not run (quoting) or rules not reloaded	Run on its own line; restart the manager
`Invalid 'if_matched_sid'` in rules test	List of IDs or undefined/forward reference	Use one ID defined earlier in the file
`grep`: Permission denied	Alert and rule files are owned by root	Use `sudo`
Counts look wrong	Looking at the last N lines only	Use `grep -c` over the whole file
Splunk shows no events	Forwarding or index name mismatch	Check the index (`sysmon`), time range and forwarder service

15.8 Cleanup and rollback

1. Restore the rules file from the backup if needed and restart the manager.
2. Turn off any extra logging you enabled (for example full archive logging) because it grows quickly.
3. Take a VM snapshot once everything works.
4. Delete test data you no longer need.

CHAPTER 16

Limitations and Future Improvements

- **No real phishing delivery.** The tests simulate the execution stage only; browser-launched shells (rule 100110) and Office-parent chains (rule 100104) were not exercised.
- **Three rules untested** (100104, 100106, 100110), and the exact effect of tuning is unmeasured.
- **Rule file not exported.** The rules in Chapter 7 are reconstructed from photographs and may contain transcription mistakes.
- **Evidence is photographic.** Many screenshots are phone photographs of a monitor, with distortion and glare. Where text was not legible, claims were limited to what could be read.
- **Internal mechanism not traced.** The reason for the missing-parent behaviour was confirmed by experiment, not with `wazuh-logtest -v`.
- **Splunk alerts not evidenced.** Only searches were captured.
- **Single endpoint, administrator shell, one short time window.** Results may differ for standard users and over time. False positives and false negatives were not measured.
- **Splunk and Wazuh share a VM,** so performance under load is untested.

Future improvements: use GoPhish for an end-to-end phishing simulation, trace rules with `wazuh-logtest -v`, export the final rules to version control, add detections for encoded PowerShell and Office parents, tune noise, and add automated rule tests that run after each change.

CHAPTER 17

Conclusion

The lab achieved its core aim: custom Wazuh rules for phishing-style execution now fire on harmless test activity, and the same events are visible and searchable in Splunk. Four rules (100105, 100107, 100111, 100510) are verified end to end in the raw alert file and in the dashboard. The most important lesson is practical: a rule that has never been seen to fire is an untested control. Here, six rules were silently inactive because they lacked the parent rule `61603`, and the correlation rule referenced rules incorrectly; both were found by checking each stage of the pipeline in turn.

The report does not claim more than the evidence shows. Three rules and the full phishing delivery chain are untested, and Splunk alerts were not saved or evidenced. The next steps are to run a GoPhish simulation, trace the rules with verbose logtest, save and test Splunk alerts, tune rule 100510, and store the final rules under version control.

Appendix A: Timeline of Work

Time (WAT)	Event	Figure
27 Oct 08:56–10:42	Earlier mshta tests (seen only in Splunk); no Wazuh alerts	29, 32
10:38–10:52	Review of `local_rules.xml` in nano; no `if_sid` present	2–5
10:43–10:50	Windows tests; Sysmon Event 1 confirmed; dashboard filtered to rule 92217	6–8
11:09	Rules test: Invalid `if_matched_sid` (comma list), rule 100107 ignored	34
11:13	`sed` command pasted incorrectly (unknown command)	35
11:16	Broad `sed` replaced all `if_matched_sid` values	36
11:20	`perl` adds `if_sid 61603` to six rules	9
11:22–11:29	100101 and 100107 repaired; manager restarted (active since 11:29:52)	10, 11
11:26	Rule 100111 alert recorded	12
11:44–11:56	Further tests; 100105 and 100510 confirmed; typing errors diagnosed	13–16
11:58–12:01	Rule 100107: count 0, definition check, count 3	17–19
12:05–12:14	Wazuh dashboard: 14 hits; alert details	20–26
16:15–17:19	Splunk searches: mshta, PowerShell	27–30
28 Oct 06:16–07:01	Splunk full table, event details, correlation search	31–33

Appendix B: Evidence Register

Table 21. Evidence register: every figure and its source (originals kept unchanged)

Figure	Title	Source file prefix (time)	Processing
Figure 1	Laboratory architecture and data flow	Drawn for report	Original diagram
Figure 2	Rules 100104 and 100105 as first written	0168da3c (10:51)	Cropped, contrast-adjusted, sharpened
Figure 3	Rules 100106 and 100107 as first written	5fbc4433 (10:51)	Cropped, contrast-adjusted, sharpened
Figure 4	Rules 100510 and 100110 as first written	9a6b46fa (10:51)	Cropped, contrast-adjusted, sharpened
Figure 5	Rules 100110 and 100111 as first written	839324d6 (10:52)	Cropped, contrast-adjusted, sharpened
Figure 6	Windows test commands and PowerShell parser errors	fc586c77 (10:46)	Cropped, contrast-adjusted, sharpened
Figure 7	Sysmon Event ID 1 present on the endpoint	91edd109 (10:50)	Cropped, contrast-adjusted, sharpened
Figure 8	Dashboard filtered to rule 92217 only	a4bebdec (10:46)	Cropped, contrast-adjusted, sharpened
Figure 9	Parent rule added to the detection rules	4c7e2e02 (11:20)	Cropped, contrast-adjusted, sharpened
Figure 10	Remaining warning after the first repairs	eb5f2ad3 (11:22)	Cropped, contrast-adjusted, sharpened
Figure 11	Manager restarted and running	83eb9efc (11:32)	Cropped, contrast-adjusted, sharpened
Figure 12	First confirmed alert from the custom rules	a196c9b0 (11:27)	Cropped, contrast-adjusted, sharpened
Figure 13	Mistyped commands that did not execute	f2d317e8 (11:45)	Cropped, contrast-adjusted, sharpened

Figure	Title	Source file prefix (time)	Processing
Figure 14	Counts of custom rules in the recent alerts	d39c5648 (11:50)	Cropped, contrast-adjusted, sharpened
Figure 15	Rule 100510 definition check	b787a062 (11:54)	Cropped, contrast-adjusted, sharpened
Figure 16	Rule 100510 count in the full alert file	20a12e1d (11:56)	Cropped, contrast-adjusted, sharpened
Figure 17	Rule 100107 count of zero before the timed test	3e6ea7a0 (11:58)	Cropped, contrast-adjusted, sharpened
Figure 18	Rule 100107 final definition	1093cae9 (11:59)	Cropped, contrast-adjusted, sharpened
Figure 19	Rule 100107 fires: count of 3	94a04de7 (12:01)	Cropped, contrast-adjusted, sharpened
Figure 20	Dashboard events table: 14 hits for the four rules	5c9da716 (12:05)	Cropped, contrast-adjusted, sharpened
Figure 21	Alert fields for rule 100105	1214031916733… (12:14)	Cropped, contrast-adjusted, sharpened
Figure 22	Alert details (1): agent, command line and directory	20874ea7 (12:08)	Cropped, contrast-adjusted, sharpened
Figure 23	Alert details (2): integrity, hashes and image	a61c6de4 (12:08)	Cropped, contrast-adjusted, sharpened
Figure 24	Alert details (3): parent process	d7f9ed03 (12:08)	Cropped, contrast-adjusted, sharpened
Figure 25	Alert details (4): Sysmon provider and raw message	20541937 (12:08)	Cropped, contrast-adjusted, sharpened
Figure 26	Alert details (5): decoder and full log	1594f9d1 (12:09)	Cropped, contrast-adjusted, sharpened
Figure 27	Splunk search for mshta.exe: 10 events over 24 hours	40339222 (Oct 27, 16:15)	Cropped, contrast-adjusted, sharpened

Figure	Title	Source file prefix (time)	Processing
Figure 28	mshta.exe results as a table	a172db43 (Oct 27, 16:29)	Cropped, contrast-adjusted, sharpened
Figure 29	mshta.exe execution times in Splunk	37b08bdf (Oct 27, 16:30)	Cropped, contrast-adjusted, sharpened
Figure 30	PowerShell launches grouped by command line	d066f32a (Oct 27, 17:16)	Cropped, contrast-adjusted, sharpened
Figure 31	All Sysmon Event 1 records: 1,197 in 24 hours	810cb846 (Oct 28, 06:21)	Cropped, contrast-adjusted, sharpened
Figure 32	Full Sysmon event in Splunk	330b6dba (Oct 28, 06:50)	Cropped, contrast-adjusted, sharpened
Figure 33	Correlation search result in Splunk	63f83b28 (Oct 28, 07:01)	Cropped, contrast-adjusted, sharpened
Figure 34	Invalid if_matched_sid reported by the rules test	9ce01652 (11:09)	Cropped, contrast-adjusted, sharpened
Figure 35	sed syntax error: unknown command '<'	31f60ded (11:13)	Cropped, contrast-adjusted, sharpened
Figure 36	Over-broad sed replaced every if_matched_sid	e16520fb (11:16)	Cropped, contrast-adjusted, sharpened

Appendix C: Command History Used in the Lab

```
# Ubuntu (user hope)
sudo nano /var/ossec/etc/rules/local_rules.xml
sudo cp /var/ossec/etc/rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml.bak
sudo grep -n -A1 '<rule id="1001' /var/ossec/etc/rules/local_rules.xml
sudo perl -0pi -e 's/(<rule id="(?:100104|100105|100106|100110|100111|100510)"[^>]*>)/$1\n
<if_sid>61603<\/if_sid>/g' /var/ossec/etc/rules/local_rules.xml
sudo sed -i '/<rule id="100101"/,/<\/rule>/s|<if_matched_sid>100111</if_matched_sid>|
<if_matched_sid>100100</if_matched_sid>|' /var/ossec/etc/rules/local_rules.xml
sudo sed -i '/<rule id="100107"/,/<\/rule>/s|<if_matched_sid>100111</if_matched_sid>|
<if_matched_sid>100105</if_matched_sid>|' /var/ossec/etc/rules/local_rules.xml
sudo /var/ossec/bin/wazuh-analysisd -t 2>&1 | grep -i 1001
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager --no-pager | head -n 5
sudo grep -E '"id":"(100105|100510|100107)"' /var/ossec/logs/alerts/alerts.json | tail -n 3
sudo tail -n 200 /var/ossec/logs/alerts/alerts.json | grep -oE '"id":"1001[0-9]+"' | sort | uniq -c
sudo grep -c '"id":"100510"' /var/ossec/logs/alerts/alerts.json
sudo grep -c '"id":"100107"' /var/ossec/logs/alerts/alerts.json
sudo /var/ossec/bin/wazuh-logtest

# Windows 10 (administrator PowerShell)
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';Id=1} -MaxEvents 5 |
Select TimeCreated, Message | Format-List
powershell.exe -Command "cd $env:USERPROFILE\Downloads; Write-Host 'Lab test v8'"
powershell.exe -Command whoami
cmd /c mshta.exe "javascript:close()"
```

Note. The command list is a consolidated record of commands used or recommended in the lab, not a verbatim shell history. Commands that failed because of syntax errors are shown in the figures, not repeated here.

Appendix D: Glossary and Notes

Term	Meaning
Agent	Software on an endpoint that collects logs and sends them to the Wazuh manager
Alert	A record written when a rule matches an event
Decoder	A set of patterns that splits a raw log into named fields
Correlation rule	A rule that fires when another rule matches several times in a time window
Integrity level	Windows label showing the privilege of a process (High means administrator)
Parent process	The process that started another process
Sourcetype / index	Splunk labels describing the kind of data and where it is stored

References. MITRE ATT&CK techniques T1218.005, T1059.001, T1566, T1566.001, T1204.001, T1204.002 (attack.mitre.org). Wazuh documentation: rules syntax (`if_sid` , `if_matched_sid` , `frequency` , `timeframe`) and `wazuh-logtest` . Splunk documentation: `streamstats` . Microsoft Sysinternals Sysmon, Event ID 1.

Appendix E: Companion Scripts

Evidence gap. These scripts consolidate commands that were used in the lab. They were assembled for this report and were **not run as a single unit**; read them before running. The rules excerpt is delivered with the report as `lab_local_rules_excerpt.xml`.

17.1 Windows test script (administrator PowerShell)

```
# Harmless lab tests. Run in an administrator PowerShell on the lab Windows VM only.
powershell.exe -Command "cd $env:USERPROFILE\Downloads; Write-Host 'Lab test'"   # rule 100111
Start-Sleep -Seconds 5
powershell.exe -Command whoami                                                    # rule 100510
Start-Sleep -Seconds 5
cmd /c mshta.exe "javascript:close()"                                             # rule 100105
Start-Sleep -Seconds 10
cmd /c mshta.exe "javascript:close()"                                             # second run < 180
s: rule 100107
```

17.2 Ubuntu verification script

```
#!/usr/bin/env bash
# Run on the Ubuntu Wazuh manager.
A=/var/ossec/logs/alerts/alerts.json
echo "== rules test (no 1001xx lines means OK) =="
sudo /var/ossec/bin/wazuh-analysisd -t 2>&1 | grep -i 1001 || echo "OK"
echo "== alert counts =="
for r in 100105 100107 100111 100510; do
  printf "%s: " "$r"; sudo grep -c "\"id\":\"$r\"" "$A"
done
```